

Requirement Analysis & Planning

1.1 Introduction

Requirement Analysis is the first and one of the most important phases of the Software Development Life Cycle (SDLC). It involves identifying business needs, understanding system requirements, and defining the overall scope of the application before development begins. A well-executed requirement analysis helps ensure that the final solution aligns with user expectations while reducing future development issues.

For the Book Now Movie Booking Application, this phase focused on understanding how movie ticket booking is managed in a theatre environment and translating those business processes into Salesforce. The application was designed to provide administrators with a centralized platform for managing movies, theatres, show schedules, customer bookings, and seat availability. In addition, the project incorporates Salesforce Agentforce to enable users to retrieve movie information through conversational AI.

Instead of manually maintaining booking records or updating seat availability, the application automates these operations using Salesforce declarative features and Apex programming. This approach improves efficiency, minimizes manual errors, and provides a structured workflow for administrators.

The outcome of this phase served as the foundation for designing the data model, configuring automation, implementing business logic, and developing the user interface throughout the project.

1.2 Problem Statement

Traditional movie booking processes often rely on manual data entry or disconnected systems to manage movies, theatres, show schedules, and customer reservations. These approaches can result in inaccurate seat availability, duplicate bookings, delayed updates, and inefficient record management.

The absence of an integrated solution also makes it difficult for administrators to monitor bookings, manage theatre schedules, and provide customers with accurate movie information. Additionally, customers increasingly expect quick access to information through conversational interfaces rather than navigating multiple screens.

To address these challenges, the Book Now Movie Booking Application was developed using Salesforce. The application centralizes movie booking operations by maintaining related business records in a single platform while automating seat management and enabling AI-assisted movie information retrieval through Agentforce.

1.3 Project Objectives

The primary objectives of the Book Now Movie Booking Application are:

- Develop a centralized movie booking management system using Salesforce.
 - Maintain movie, theatre, show, and booking information using custom objects.
 - Establish relationships between business entities through lookup relationships.
 - Enforce business rules using validation rules and formula fields.
 - Automatically update seat availability after successful bookings using Apex.
 - Provide an intuitive Lightning user interface for administrators.
 - Generate reports and dashboards for operational monitoring.
 - Integrate Salesforce Agentforce to provide AI-powered movie information.
 - Ensure data accuracy, scalability, and maintainability through Salesforce best practices.
-

1.4 Project Scope

The scope of this project includes the design, development, and testing of a Salesforce-based application that manages the complete movie booking workflow.

The application supports the following business functions:

- Managing movie information.
- Maintaining theatre details.
- Scheduling movie shows.
- Recording customer bookings.
- Tracking available seats.
- Validating booking data.
- Automating seat updates through Apex.
- Retrieving movie information using Agentforce.
- Monitoring application data through reports and dashboards.

The project is intended as a demonstration of Salesforce Administration, Apex Development, and Agentforce capabilities within a single integrated solution. Features such as online payments, customer self-service portals, and SMS notifications are outside the scope of the current implementation but may be considered in future enhancements.

1.5 Functional Requirements

The functional requirements define the core operations that the Book Now Movie Booking Application must perform to support movie booking management. These requirements guided the development of the application and ensured that all essential business processes were implemented within Salesforce.

- **Functional Requirements**

Category	Requirement Definition
Movie Management	Create and manage movie records.
Theatre Management	Maintain theatre information.
Show Scheduling	Schedule movie shows by associating movies with theatres.
Booking Management	Record customer bookings for selected shows.
Data Validation	Validate booking information before saving records.
Automation	Automatically update available seats after each successful booking.

Data Insights	Display calculated values using formula fields.
Reporting	Generate reports and dashboards for monitoring business operations.
AI Integration	Allow users to retrieve movie information through Salesforce Agentforce.



1.6 Non-Functional Requirements

In addition to functional capabilities, the application was designed to satisfy several non-functional requirements that improve reliability, usability, and maintainability.

Non-Functional Requirements

Category	Requirement Definition
Usability	Provide an intuitive and user-friendly interface.
Data Integrity	Maintain data accuracy and integrity.
Performance	Respond efficiently during record operations.
Scalability	Support future enhancements and scalability.
Security	Protect business data using Salesforce security features.
Reliability	Ensure reliable execution of automation and Apex logic.



1.7 System Requirements

The development and testing of the Book Now Movie Booking Application were carried out using the following software environment

Technical Specifications

Component	Specification
Platform	Salesforce Lightning Platform
Development Environment	Salesforce Developer Edition
Programming Language	Apex
Automation Tool	Salesforce Flow
AI Platform	Salesforce Agentforce
Browser	Google Chrome / Microsoft Edge
Operating System	Windows 10/11

1.8 Development Methodology

The project followed a structured development approach consisting of requirement analysis, backend configuration, user interface development, security implementation, testing, and documentation. Each phase was completed sequentially to ensure that every component was properly designed, implemented, and validated before proceeding to the next stage.

This methodology helped maintain consistency throughout the project while ensuring that business requirements were successfully translated into a working Salesforce application.

1.11 Project Constraints

Like any software development project, the Book Now application was developed under certain technical and operational constraints.

The major constraints include:

Project Constraints

<i>Constraint</i>	<i>Description</i>
<i>Developer Edition Limitations</i>	<i>The project was developed using a Salesforce Developer Edition Org, which imposes limitations on storage, user licenses, API calls, and deployment capabilities.</i>
<i>Internship Timeline</i>	<i>The implementation was completed within the internship schedule, limiting the inclusion of advanced enterprise-level features.</i>
<i>Limited External Integration</i>	<i>No third-party APIs or payment gateways were integrated because the primary objective was learning Salesforce CRM development rather than building a production-ready commercial application.</i>

Sample Data	<i>Movie records, theatre information, and booking details were created using sample data for demonstration and testing purposes.</i>
--------------------	---

1.12 Project Deliverables

Upon successful completion, the project produces the following deliverables:

- **Project Deliverables**

Category	Deliverables
Technical Deliverables	• Salesforce Developer Org Configuration • Custom Objects • Custom Fields • Object Relationships • Validation Rules • Formula Fields • Lightning Application • Apex Trigger • Apex Handler Class • Apex Test Class • Agentforce AI Agent • User Profiles • Roles and Security Configuration
Documentation Deliverables	• Software Requirement Documentation • Technical Design Documentation • Testing Report • User Guide • Salesforce Screenshots • Final Project Report • GitHub Repository

1.14 Chapter Summary

This chapter established the business foundation for the Book Now Movie Booking Application by identifying the problem domain, defining project objectives, analyzing stakeholder requirements, and outlining the functional and non-functional expectations of the system. It also described the project's scope, assumptions, constraints, execution strategy, and expected deliverables.

A well-defined planning phase ensures that the subsequent stages of Salesforce development align with the intended business goals. The insights gathered during this chapter guided the design of the data model, automation logic, user interface, and security framework implemented

in later phases of the project.

Backend Development & Salesforce Configuration

2.1 Introduction

The Backend Development phase represents the core implementation stage of the Book Now Movie Booking Application. During this phase, the business requirements identified in the planning stage were transformed into a fully functional Salesforce application through a combination of declarative configuration and programmatic development.

Unlike traditional software development where backend logic is primarily written using programming languages, Salesforce offers a hybrid development model. Business requirements can be implemented using configuration-based features such as Custom Objects, Validation Rules, Flows, and Formula Fields, while more complex business logic is handled using Apex Classes and Triggers.

For the Book Now application, the backend was designed to ensure that movie information, theatre details, show schedules, and customer bookings are managed efficiently while maintaining data integrity throughout the booking process. Automation features were incorporated to reduce manual intervention, improve consistency, and enforce business rules.

This chapter explains the complete backend architecture, object design, field configuration, automation logic, Apex implementation, and Agentforce integration that collectively form the foundation of the application.

2.2 Salesforce Development Environment

The Book Now Movie Booking Application was developed within a Salesforce Developer Edition organization, providing a cloud-based platform for designing, configuring, and testing CRM applications.

The development environment supports both declarative tools and Apex programming, allowing rapid implementation of business requirements while minimizing infrastructure management.

The following Salesforce capabilities were extensively utilized throughout the project:

Application Features and Purposes

Feature	Purpose
Custom Objects	Store business data such as Movies, Theatres, Shows, and Bookings.
Custom Fields	Capture detailed information for each business entity.
Lookup Relationships	Establish relationships between objects.
Validation Rules	Prevent invalid booking information.
Formula Fields	Automatically calculate values such as total ticket price.
Lightning App Builder	Develop an intuitive user interface.
Apex Classes	Implement advanced booking logic.
Apex Triggers	Automatically update seat availability.
Agentforce	Provide AI-powered movie information assistance.

Reports & Dashboards	Generate business insights and analytics.
----------------------	---

The Salesforce platform provides an ideal environment for this application due to its built-in security model, automation capabilities, scalability, and cloud infrastructure.

2.3 Data Model Design

The data model forms the foundation of the Book Now Movie Booking Application by organizing business information into logical and related entities. A well-designed data model improves data consistency, reduces duplication, and simplifies application maintenance.

The application consists of four primary custom objects:

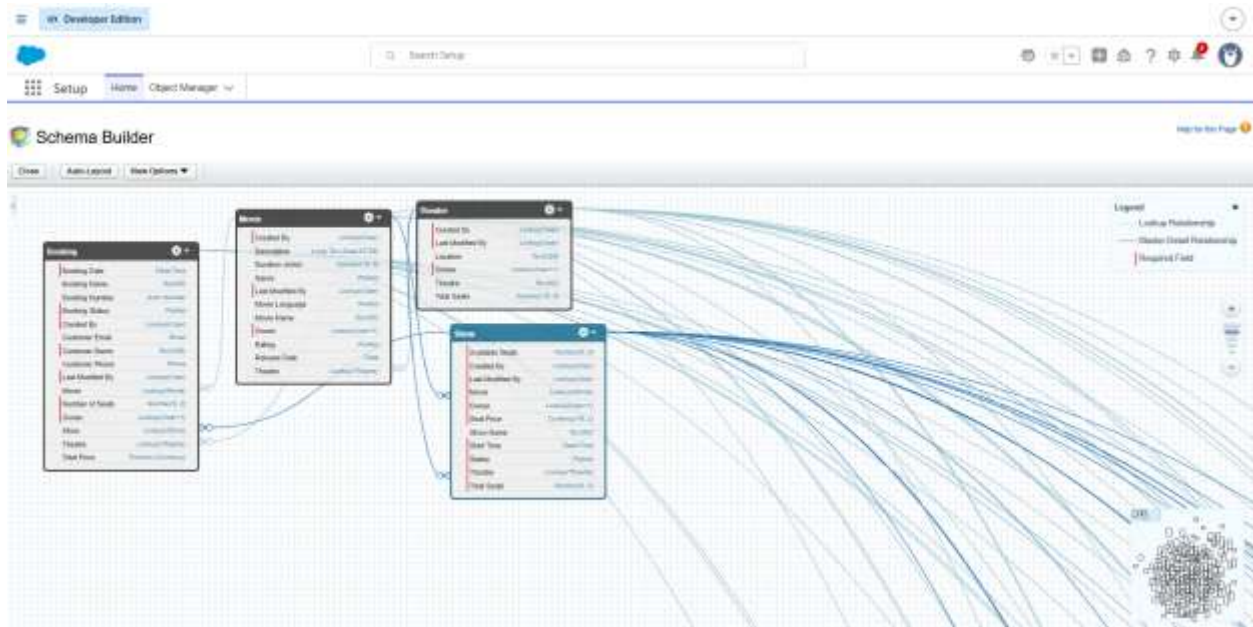
- Movie
- Theatre
- Show
- Booking

These objects are connected using **Lookup Relationships**, allowing administrators to manage movies, theatres, show schedules, and customer bookings efficiently. This modular design supports accurate reporting, simplifies record management, and allows future enhancements without major structural changes.

Implementation Outcome

- Eliminates duplicate business data.
- Improves data consistency.
- Supports multiple theatres and movie shows.
- Simplifies report generation.
- Provides a scalable application structure.

Figure 2.3 – Data Model Diagram



2.4 Custom Object Development

The Book Now Movie Booking Application is built using four custom Salesforce objects that represent the core business entities involved in movie booking. Each object was designed to store a specific category of information while maintaining relationships with other objects through Lookup Relationships.

Separating business data into individual objects follows Salesforce best practices by improving maintainability, minimizing data redundancy, and enabling efficient record management. The following sections describe each custom object, its purpose, key fields, relationships, and implementation.

2.4.1 Movie Object

Purpose

The **Movie** object serves as the central repository for storing movie-related information within the Book Now Movie Booking Application. It maintains details such as the movie name, genre, language, duration, release date, rating, and description. By storing this information in a single location, multiple shows can reference the same movie record, ensuring consistency and eliminating duplicate data.

This centralized approach simplifies movie management, enables efficient scheduling of shows, and allows Agentforce to retrieve accurate movie information whenever requested by users.

Major Fields: Movie Object

Field Name	Description
Movie Name	Stores the name of the movie.
Genre	Specifies the movie category.
Language	Stores the language of the movie.
Duration	Indicates the running time of the movie.
Release Date	Stores the official release date.
Rating	Stores the audience rating.
Description	Contains a brief synopsis of the movie.

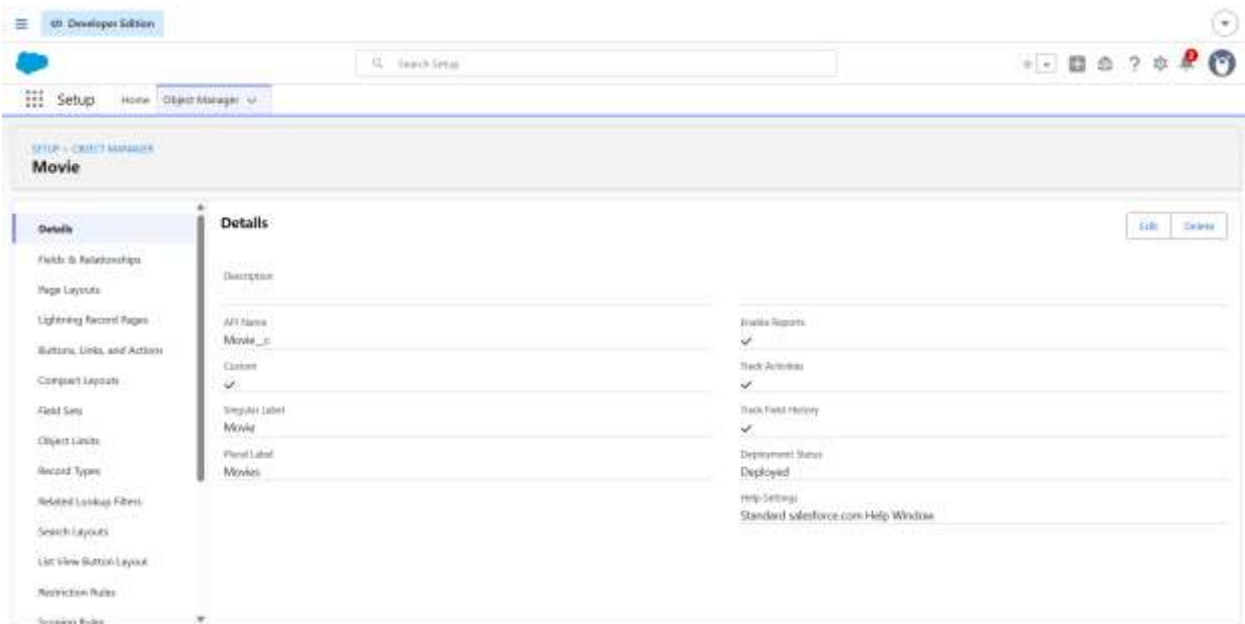
Relationships

The Movie object maintains a **Lookup Relationship** with the Show object. This relationship allows a single movie to be associated with multiple shows scheduled across different theatres while ensuring that all show records reference a single source of movie information.

Implementation Outcome

- Centralized movie management.
 - Reduced duplication of movie information.
 - Simplified movie updates.
 - Support for multiple shows of the same movie.
 - Improved reporting accuracy.
 - Seamless integration with Agentforce for movie information retrieval.
-

Figure 2.4 – Movie Custom Object



Screenshot Description: Displays the Movie custom object configuration.

Figure 2.5 – Movie Object Fields

Movie				
Details	Fields & Relationships 11 Items, Sorted by Field Label			
Fields & Relationships	FIELD LABEL	FIELD NAME	DATA TYPE	INDEXED
Page Layouts	Created By	CreatedBy	Lookup(User)	
Lightning Record Pages	Description	Description__c	Long Text Area (HTML)	
Buttons, Links, and Actions	Duration (Hours)	Duration_hours__c	Number(16, 0)	
Compact Layouts	Genre	Genre__c	Picklist	
Field Sets	Last Modified By	LastModifiedBy	Lookup(User)	
Object Limits	Movie Language	Movie_Language__c	Picklist	
Record Types	Movie Name	Name	Text(80)	✓
Related Lookup Filters	Owner	OwnerId	Lookup(Owner Group)	✓
Select Layouts	Rating	Rating__c	Picklist	
List View Button Layout	Release Date	Release_Date__c	Date	
Restriction Rules	Theatre	Theatre__c	Lookup(Theatre)	✓
Scoping Rules				
Object Access				
Field Access				

Screenshot Description: Displays the custom fields created for the Movie object.

2.4.2 Theatre Object

Purpose

The **Theatre** object stores information about theatres where movies are screened. It maintains essential details such as the theatre name, location, seating capacity, and operational status. By storing theatre information as a separate custom object, the application avoids data duplication and supports efficient management of multiple theatres.

Each theatre can host multiple movie shows through its relationship with the Show object, providing a scalable structure that can easily accommodate future expansion. This design also simplifies theatre maintenance without affecting existing booking records..

Business Requirement

The business required a centralized mechanism for managing all theatres participating in the movie booking process. Theatre administrators should be able to register multiple theatres, maintain their seating capacities, update addresses when required, and associate each theatre with multiple movie shows.

By separating theatre information into an independent object, the system supports expansion into multiple cities and locations while maintaining data consistency.

Design Considerations

Several design principles were considered during the implementation of the Theatre object:

- Theatre information should only be stored once.
- A single theatre should support multiple movie shows.
- Future expansion to multiple branches should not require structural changes.
- Booking records should reference theatres indirectly through show schedules.

This normalized design improves reporting and ensures that updates made to theatre information are automatically reflected wherever the theatre is referenced.

Relationships

The Theatre object maintains a **Lookup Relationship** with the **Show** object. A single theatre can be associated with multiple shows, while each show belongs to one theatre. This relationship simplifies scheduling, reporting, and booking management.

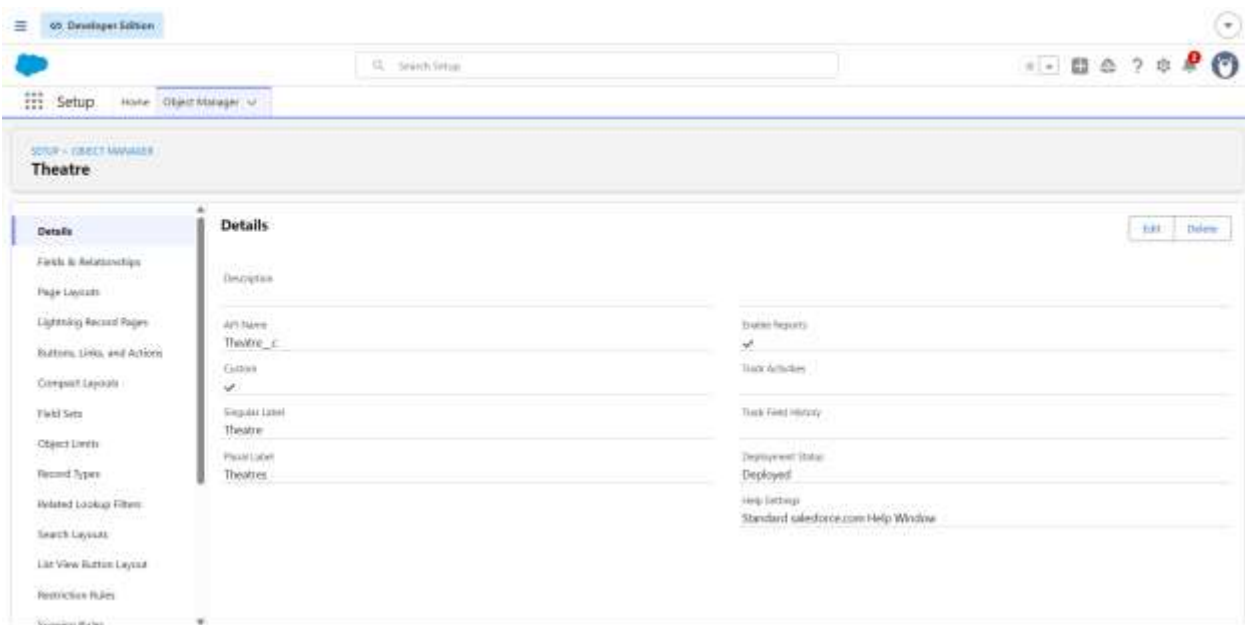
Implementation Outcome

- Centralized management of theatre information.
- Reduced duplication of theatre records.
- Simplified scheduling of movie shows.
- Improved reporting and analytics.
- Easy scalability for additional theatre branches.
- Consistent and organized business data.

Expected Output

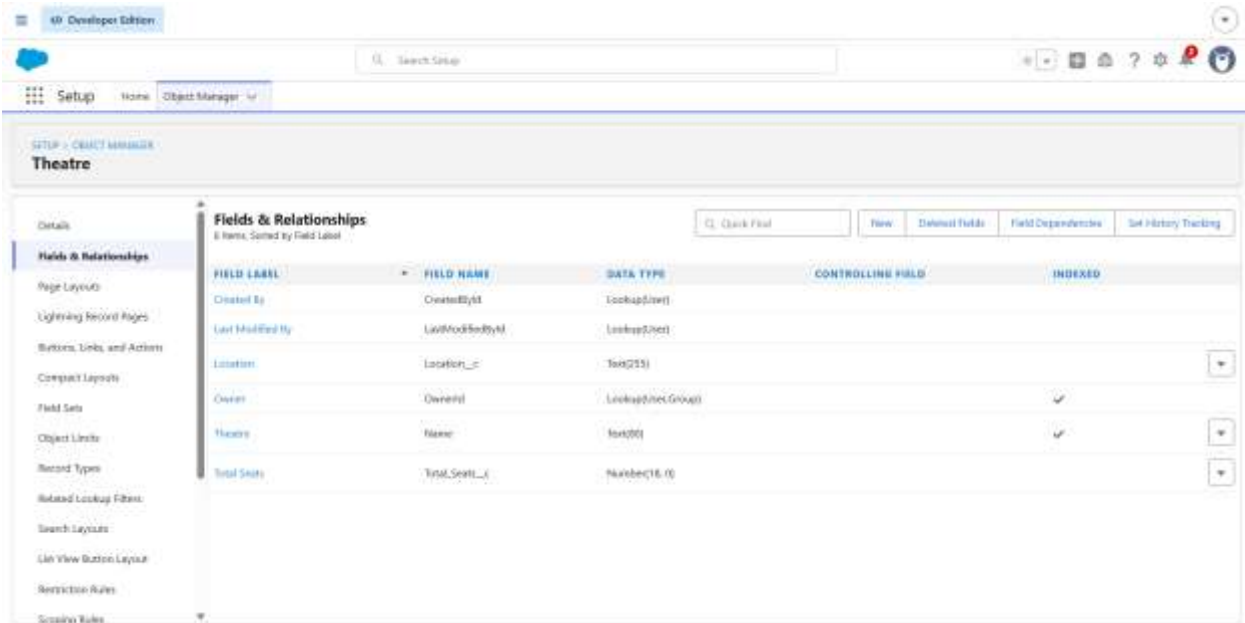
After implementation, administrators should be able to create, modify, and manage theatre records independently. All movie shows created within the application should reference an existing theatre, ensuring accurate scheduling and booking management.

Figure 2.6 – Theatre Custom Object



Screenshot Description: Displays the Theatre custom object configuration page.

Figure 2.7 – Theatre Object Fields



FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedByM	Lookup(User)		
Last Modified By	LastModifiedByM	Lookup(User)		
Location	Location_c	Text(255)		
Owner	OwnerId	Lookup(OwnerGroup)		✓
Name	Name	Text(255)		✓
Total Seats	TotalSeats_c	Number(16, 0)		

Screenshot Description: Displays the custom fields created for the Theatre object.

2.4.3 Show Object

Purpose

The **Show** object manages the scheduling of movies by linking the **Movie** and **Theatre** objects. It stores dynamic information such as show date, show time, ticket price, total seats, and available seats, enabling administrators to organize movie screenings efficiently.

Each show represents a specific screening of a movie at a particular theatre and serves as the primary reference for customer bookings. This design supports multiple screenings of the same movie across different theatres while maintaining accurate scheduling and seat availability.

Relationships

The Show object serves as the central link between the application's master and transaction records.

- References one **Movie**.
- References one **Theatre**.
- Is referenced by multiple **Booking** records.

This relationship structure ensures accurate scheduling, efficient navigation between records, and reliable reporting.

Implementation Outcome

- Flexible scheduling of movie shows.
- Support for multiple screenings across theatres.
- Centralized management of ticket pricing.
- Automated seat availability tracking.
- Improved operational planning and reporting.
- Better customer experience through accurate scheduling.

Figure 2.9 – Show Custom Object

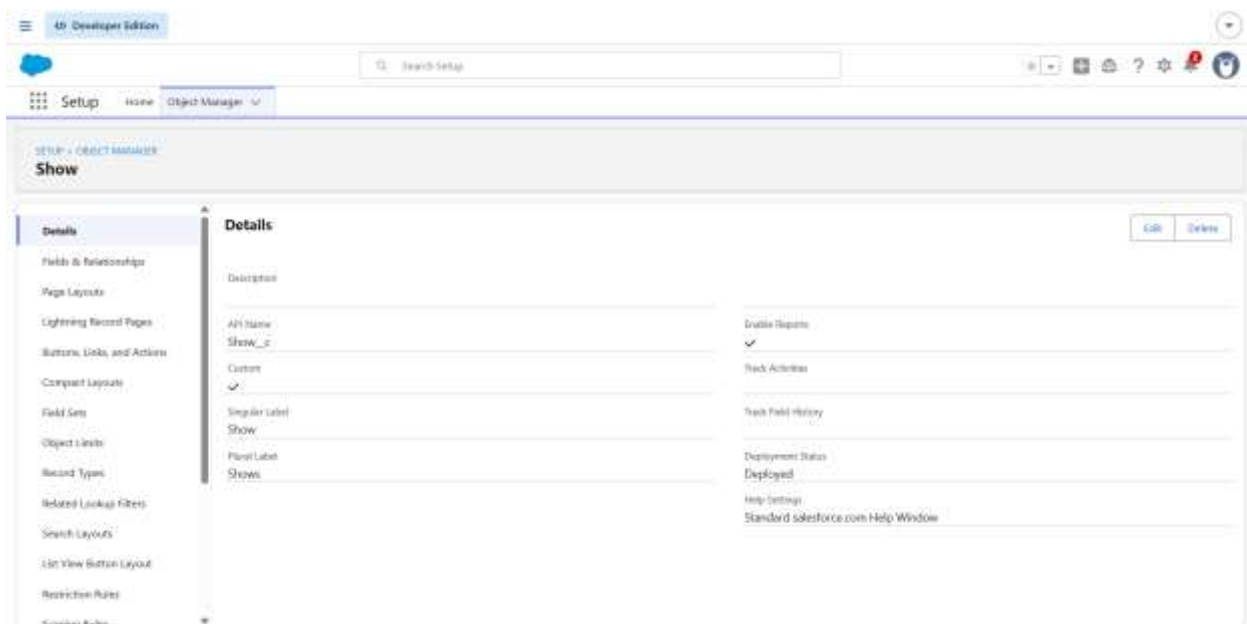


Figure 2.10 – Show Object Fields

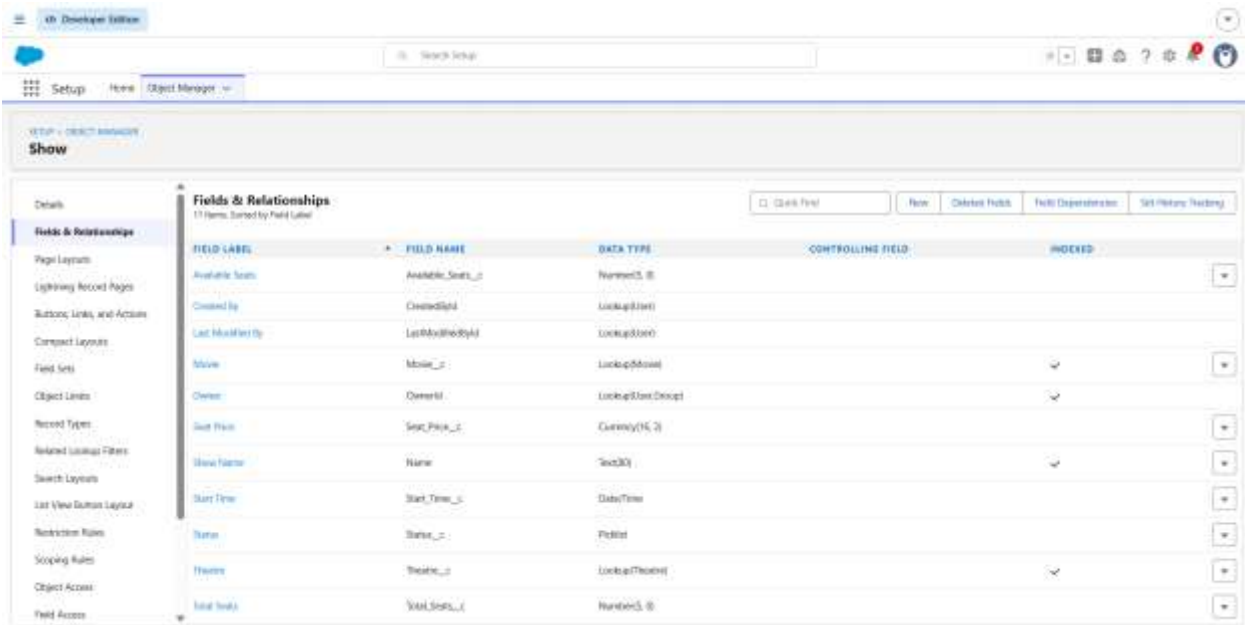
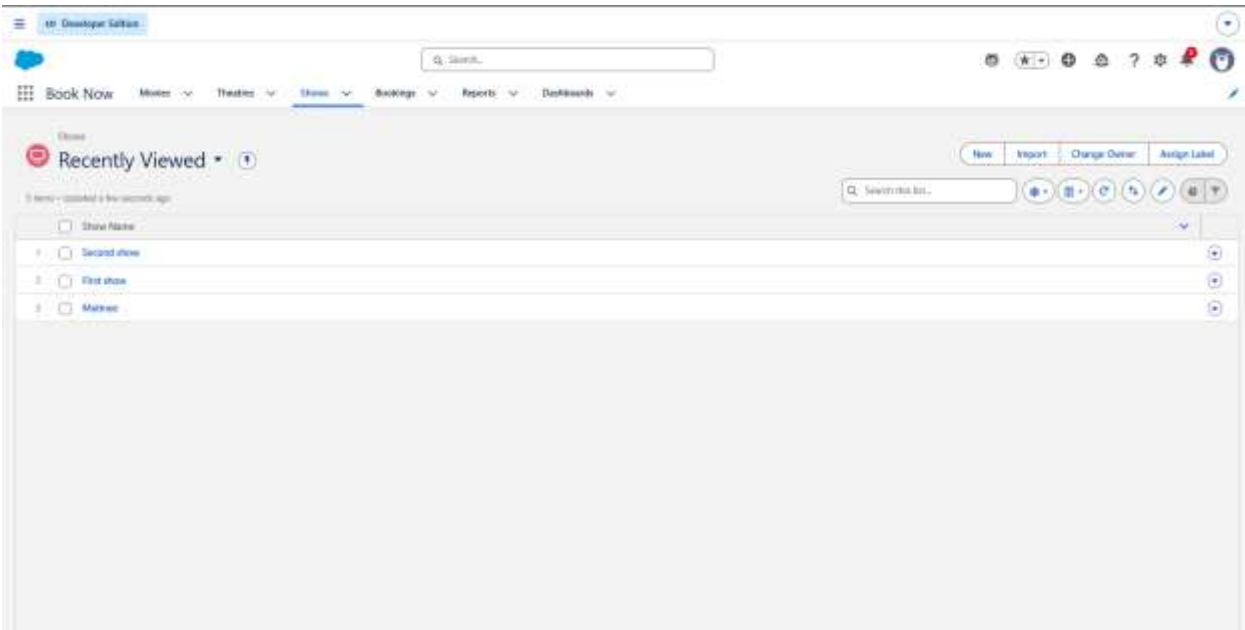


Figure 2.11 – Show Records



2.4.4 Booking Object

Purpose

The **Booking** object is the primary transaction object of the Book Now Movie Booking Application. It records every customer reservation by storing booking details such as customer information, selected show, number of seats, booking date, and total ticket amount. Unlike the Movie, Theatre, and Show objects, which store master data, the Booking object captures the actual business transactions performed within the application.

The Booking object also works with Apex automation to validate booking information and automatically update seat availability after each successful reservation, ensuring accurate and reliable booking management.

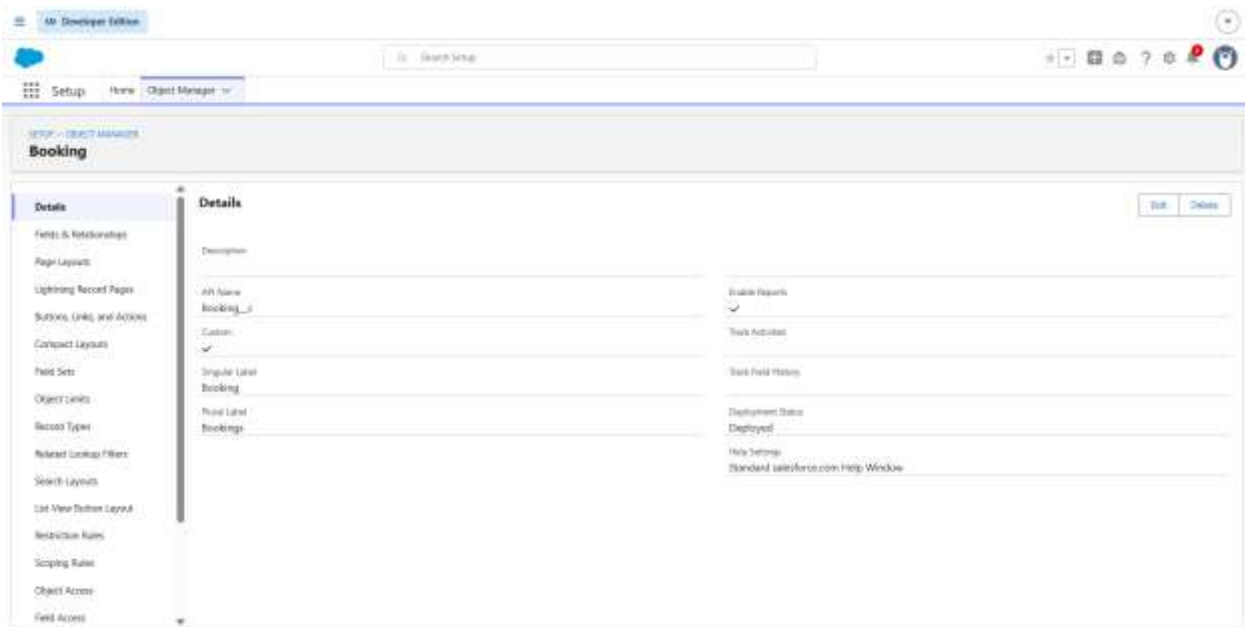
Relationships

The Booking object maintains a **Lookup Relationship** with the **Show** object. Through this relationship, each booking is indirectly associated with the corresponding Movie and Theatre, enabling efficient navigation, reporting, and seat management.

Implementation Outcome

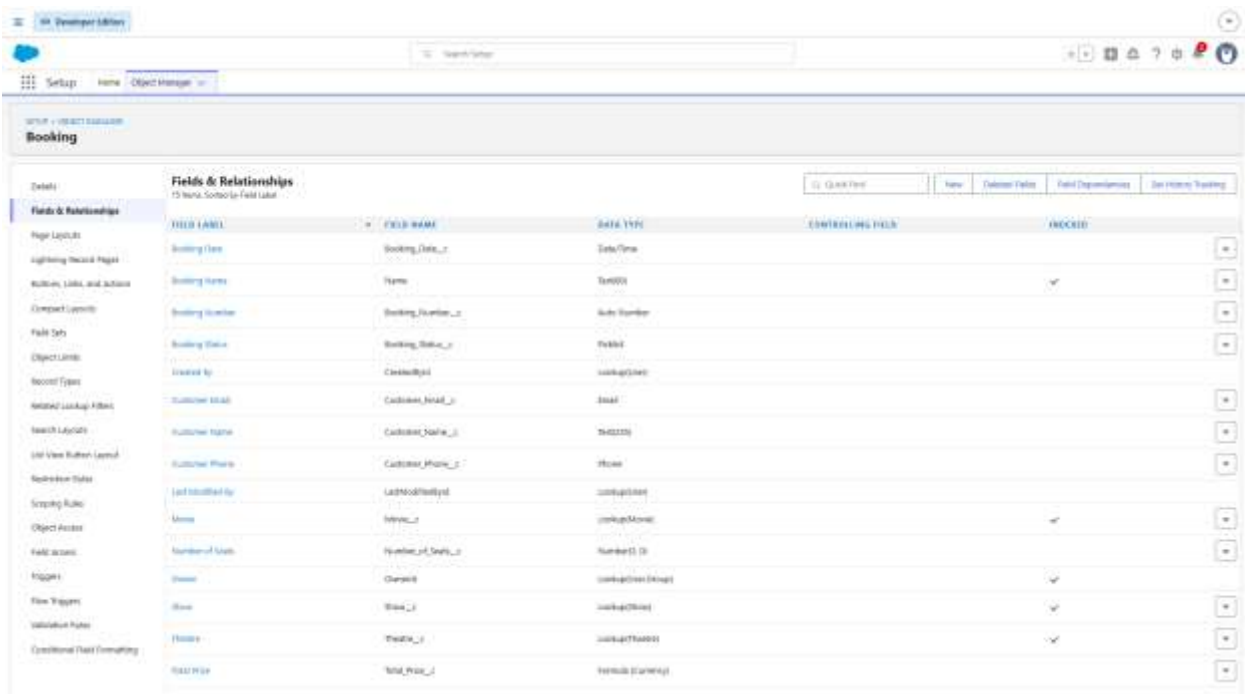
- Centralized management of customer bookings.
 - Accurate tracking of ticket reservations.
 - Automatic seat availability updates through Apex.
 - Improved customer record management.
 - Enhanced reporting and analytics.
 - Reduced manual effort through automation.
-

Figure 2.12 – Booking Custom Object



Screenshot Description: Displays the Booking custom object configuration.

Figure 2.13 – Booking Object Fields



Screenshot Description: Displays the custom fields created for the Booking object.

2.5 Object Relationships

Introduction

Object relationships are essential for maintaining a structured and connected data model in Salesforce. In the Book Now Movie Booking Application, **Lookup Relationships** were implemented to establish logical connections between the Movie, Theatre, Show, and Booking objects. These relationships ensure data consistency, minimize duplication, and support efficient reporting and automation.

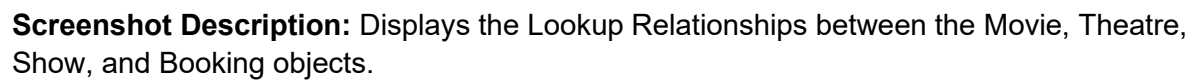
Data Model Relationships

Parent Object	Child Object	Relationship Type	Purpose
Movie	Show	One-to-Many (Lookup)	A movie can have multiple scheduled shows.
Theatre	Show	One-to-Many (Lookup)	A theatre can host multiple movie shows.
Show	Booking	One-to-Many (Lookup)	A show can have multiple customer bookings.

Implementation Outcome

- Maintains data consistency across related records.
- Eliminates duplicate data.

- ### Figure 2.15 – Object Relationship Diagram



2.6 Custom Field Configuration

Introduction

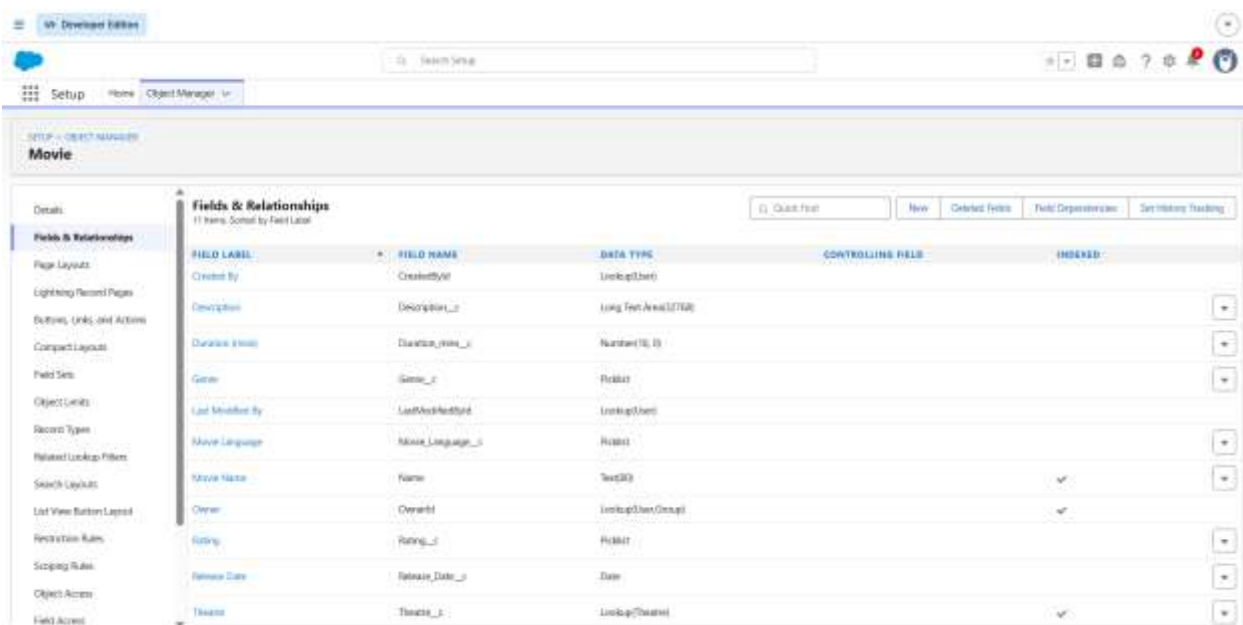
Custom fields were created to store business-specific information required for managing movies, theatres, shows, and customer bookings. Appropriate field data types such as Text, Number, Date, Currency, Picklist, Formula, and Lookup were selected to ensure accurate data entry, maintain consistency, and support automation throughout the application.

The custom fields were configured based on the business requirements and play a vital role in record management, reporting, and Apex automation.

Movie Object Fields

Custom fields were created to store movie details such as name, genre, language, duration, release date, rating, and description. These fields provide complete information about each movie and support Agentforce in retrieving movie details.

Figure 2.16 – Movie Custom Fields

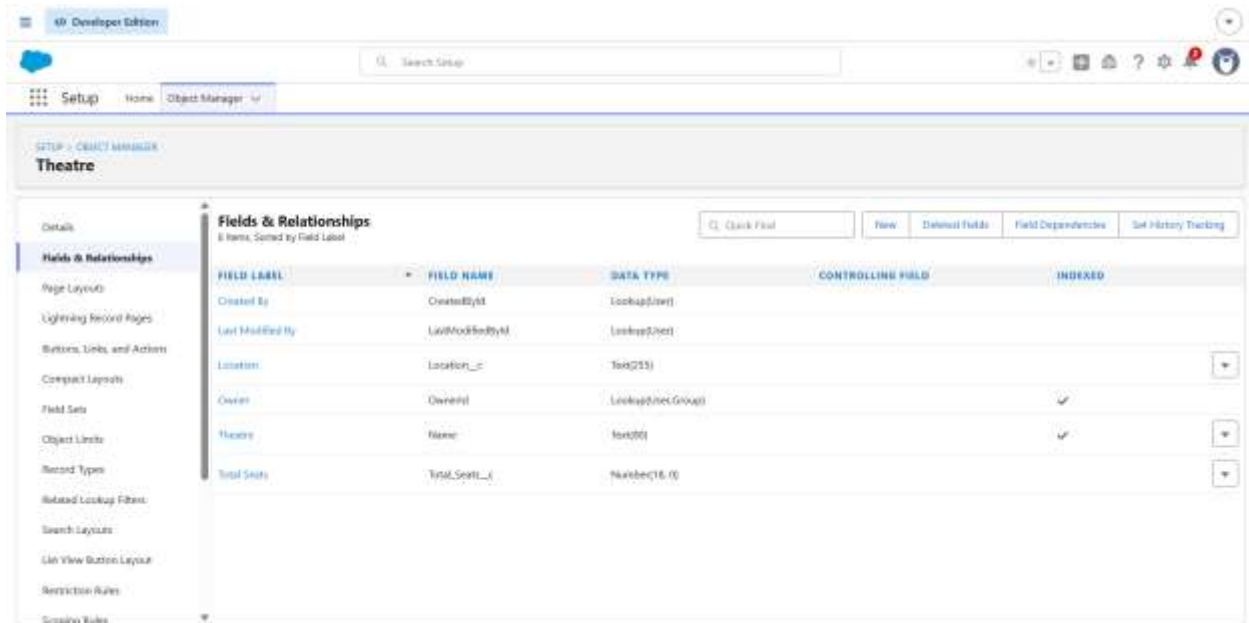


FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedBy	Lookup(User)		
Description	Description__c	Long Text Area (HTML)		
Duration (mins)	Duration_mins__c	Number(16, 0)		
Genre	Genre__c	Picklist		
Last Modified By	LastModifiedBy	Lookup(User)		
Movie Language	Movie_Language__c	Picklist		
Movie Name	Name	Text(30)		✓
Owner	OwnerId	Lookup(User/Group)		✓
Rating	Rating__c	Picklist		
Release Date	Release_Date__c	Date		
Theatre	Theatre__c	Lookup(Theatre)		✓

Theatre Object Fields

The Theatre object contains custom fields for storing theatre information, including theatre name, location, seating capacity, and operational status. These fields help administrators efficiently manage multiple theatres.

Figure 2.17 – Theatre Custom Fields



FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Created By	CreatedBy	Lookup(User)		
Last Modified By	LastModifiedBy	Lookup(User)		
Location	Location__c	Text(255)		
Owner	OwnerId	Lookup(RoleGroup)		✓
Theatre	Name	Text(200)		✓
Total Seats	TotalSeats__c	Number(16, 0)		

Show Object Fields

The Show object includes fields for scheduling movies, assigning theatres, managing ticket prices, and tracking seat availability. These fields form the core of the booking process.

Figure 2.18 – Show Custom Fields

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Available Seats	Available_Seats__c	Number(3, 0)		
Created By	CreatedBy	Lookup(8char)		
Last Modified By	LastModifiedBy	Lookup(8char)		
Movie	Movie__c	Lookup(55char)		✓
Owner	OwnerId	Lookup(18char/lookup)		✓
Seat Price	Seat_Price__c	Currency(15, 2)		
Show Name	Name	Text(30)		✓
Start Time	Start_Time__c	Date/Time		
Status	Status__c	Picklist		
Theatre	Theatre__c	Lookup(75char)		✓
Total Seats	TotalSeats__c	Number(5, 0)		

Booking Object Fields

The Booking object stores customer reservation details, including customer information, selected show, number of seats, booking date, and total booking amount. These fields support booking management and reporting.

Figure 2.19 – Booking Custom Fields

FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Booking Date	Booking_Date__c	Date/Time		
Booking Name	Name	Text(50)		✓
Booking Number	Booking_Number__c	Auto Number		
Booking Status	Booking_Status__c	Picklist		
Created By	CreatedBy	Lookup(8char)		
Customer Email	Customer_Email__c	Email		
Customer Name	Customer_Name__c	Text(255)		
Customer Phone	Customer_Phone__c	Phone		
Last Modified By	LastModifiedBy	Lookup(8char)		
Movie	Movie__c	Lookup(55char)		✓
Number of Seats	Number_of_Seats__c	Number(5, 0)		
Show	Show__c	Lookup(55char/lookup)		✓
Seat	Seat__c	Lookup(8char)		✓
Theatre	Theatre__c	Lookup(75char)		✓
Total Price	Total_Price__c	Currency(15/lookup)		

2.7 Lookup Relationship Configuration

Introduction

Lookup Relationships were configured to establish logical connections between the custom objects in the Book Now Movie Booking Application. These relationships ensure data consistency, minimize duplication, and enable efficient navigation between related records while supporting automation and reporting.

Configured Lookup Relationships

Relationship Business Purposes

Relationship	Business Purpose
Movie → Show	Allows one movie to be scheduled for multiple shows.
Theatre → Show	Enables a theatre to host multiple movie shows.
Show → Booking	Associates each booking with a specific movie show and supports automatic seat management.

Implementation Outcome

- Eliminates duplicate data.
- Improves data consistency.
- Simplifies record management.
- Supports reports and dashboards.
- Enables Apex automation and Salesforce Flows.
- Provides a scalable application structure.

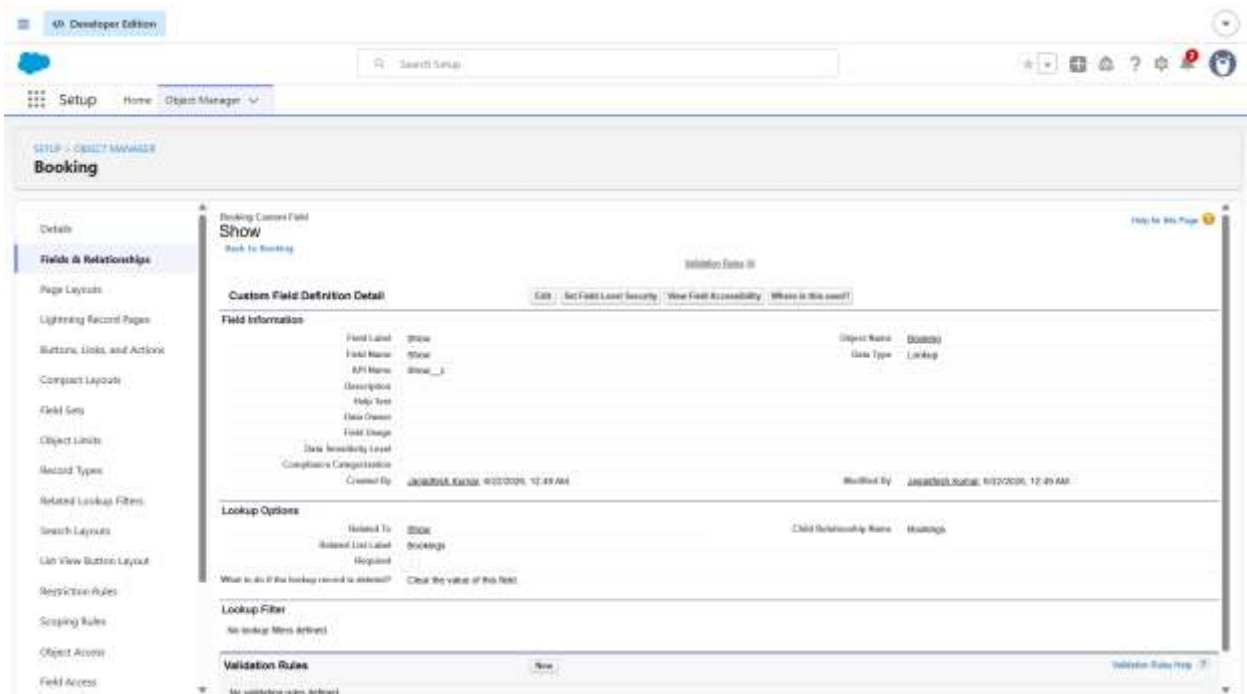
Figure 2.20 – Movie Lookup Relationship

The screenshot shows the Salesforce Setup interface for a Custom Field Definition. The left sidebar contains navigation links: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, List View Button Layout, Record Rules, Scoping Rules, Object Access, and Field Access. The main content area is titled 'Movie' and includes a 'Back to Home' link. It features tabs for 'Custom Field Definition Detail', 'Edit', 'Set Field-Level Security', 'View Field Accessibility', and 'Where is this used?'. The 'Field Information' section shows: Field Label (Movie), Field Name (movie), API Name (movie__c), Description, Help Text, Data Category, Field Design, Data Accessibility Level, Compliance Categorization, and Created By (Jagadeesh Kumar, 4/23/2020, 10:36 AM). The 'Lookup Options' section shows: Related To (Movie), Related List Label (Required), and a checkbox 'What to do if the lookup record is deleted?' (checked). The 'Lookup Filter' section shows 'No lookup filter defined'. The 'Validation Rules' section shows 'No validation rules defined'. The 'Child Relationship Name' is 'movie'.

Figure 2.21 – Theatre Lookup Relationship

The screenshot shows the Salesforce Setup interface for a Custom Field Definition. The left sidebar contains navigation links: Details, Fields & Relationships (selected), Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, List View Button Layout, Record Rules, Scoping Rules, Object Access, and Field Access. The main content area is titled 'Theatre' and includes a 'Back to Home' link. It features tabs for 'Custom Field Definition Detail', 'Edit', 'Set Field-Level Security', 'View Field Accessibility', and 'Where is this used?'. The 'Field Information' section shows: Field Label (Theatre), Field Name (theatre), API Name (theatre__c), Description, Help Text, Data Category, Field Design, Data Accessibility Level, Compliance Categorization, and Created By (Jagadeesh Kumar, 4/23/2020, 10:37 AM). The 'Lookup Options' section shows: Related To (Theatre), Related List Label (Required), and a checkbox 'What to do if the lookup record is deleted?' (checked). The 'Lookup Filter' section shows 'No lookup filter defined'. The 'Validation Rules' section shows 'No validation rules defined'. The 'Child Relationship Name' is 'theatre'.

Figure 2.22 – Booking Lookup Relationship



2.8 Custom Tabs Configuration

Introduction

Custom tabs were created to provide quick and organized access to the custom objects within the Book Now Movie Booking Application. These tabs enable administrators to efficiently manage movies, theatres, shows, and bookings directly from the Lightning Application, improving navigation and user experience.

Configured Custom Tabs

Custom Tab	Purpose
Movie	Manage movie records.
Theatre	Manage theatre information.
Show	Schedule and manage movie shows.
Booking	Create and monitor customer bookings.

Figure 2.24 – Movie Tab

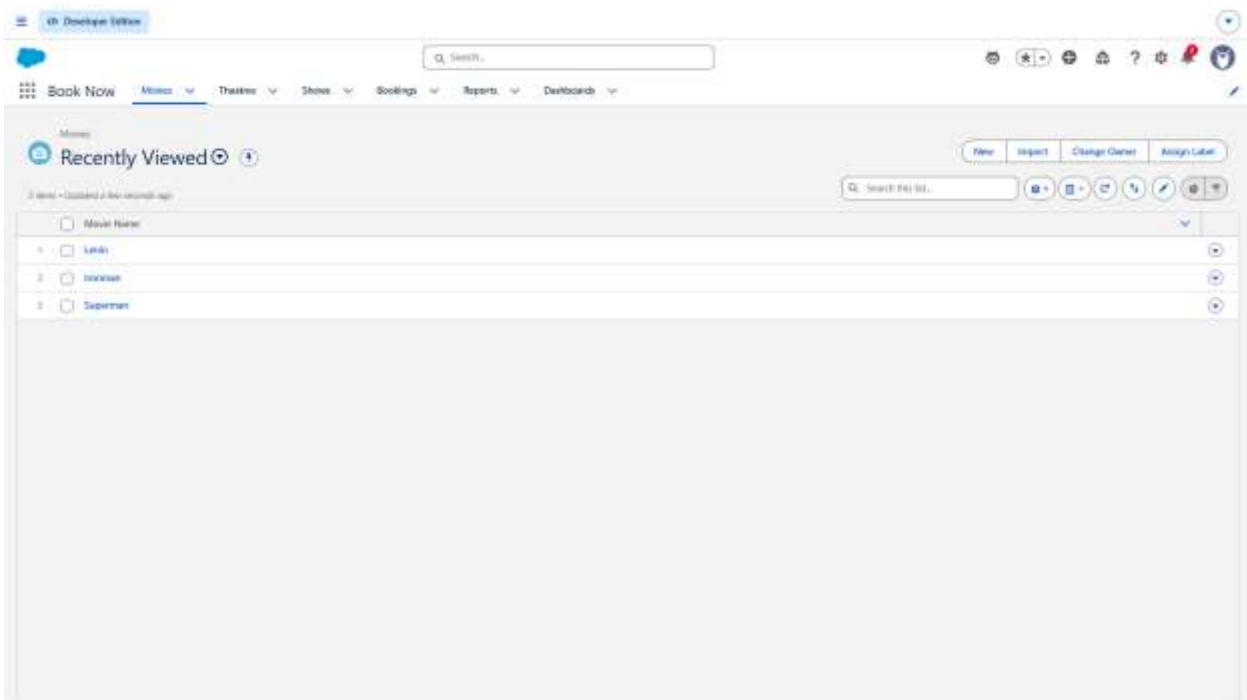


Figure 2.25 – Theatre Tab

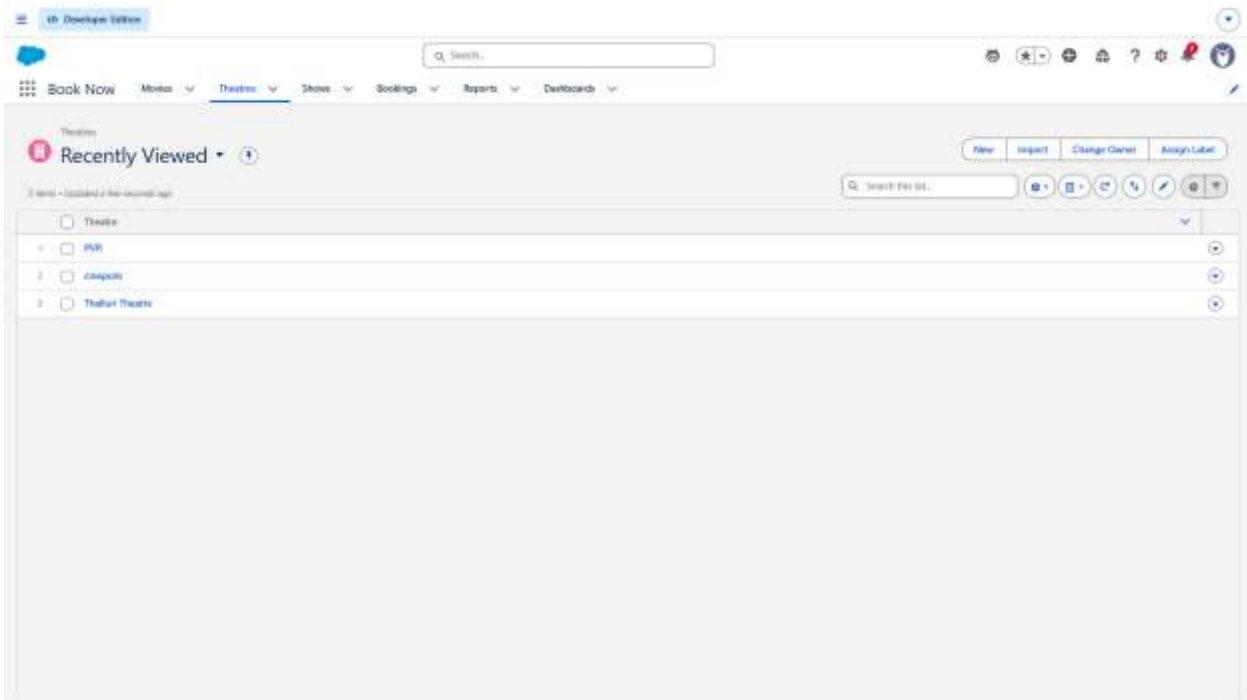


Figure 2.26 – Show Tab

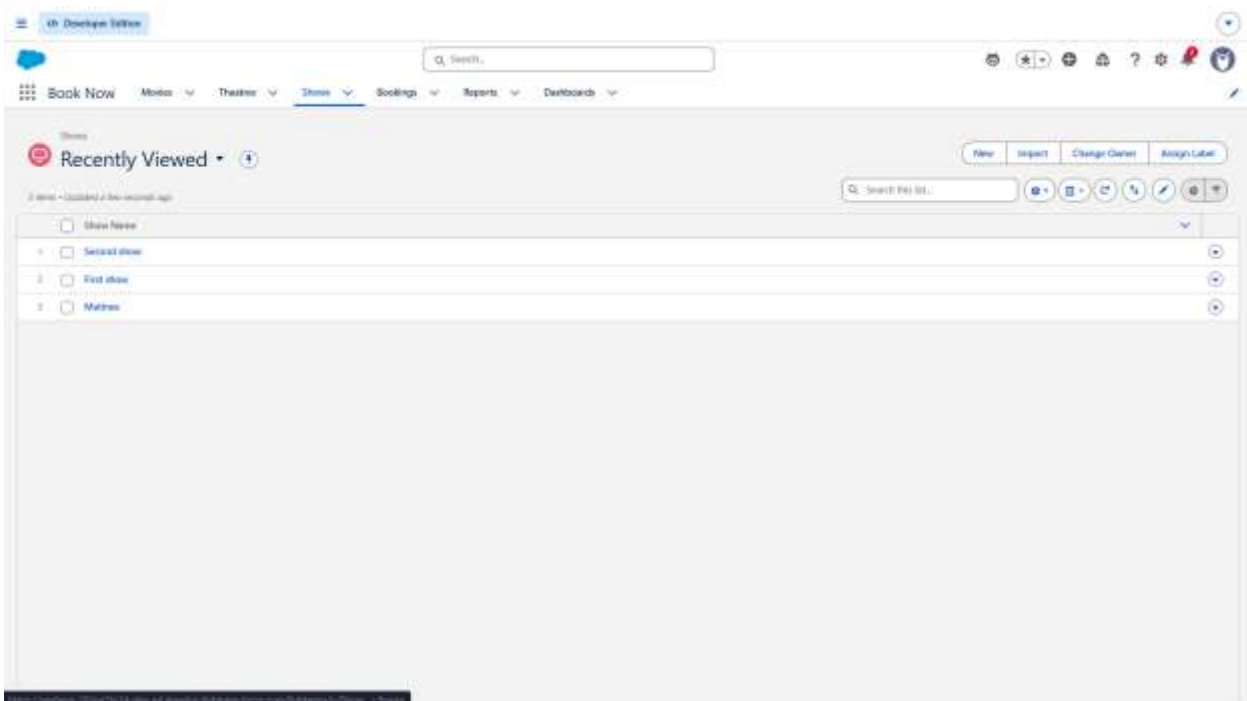
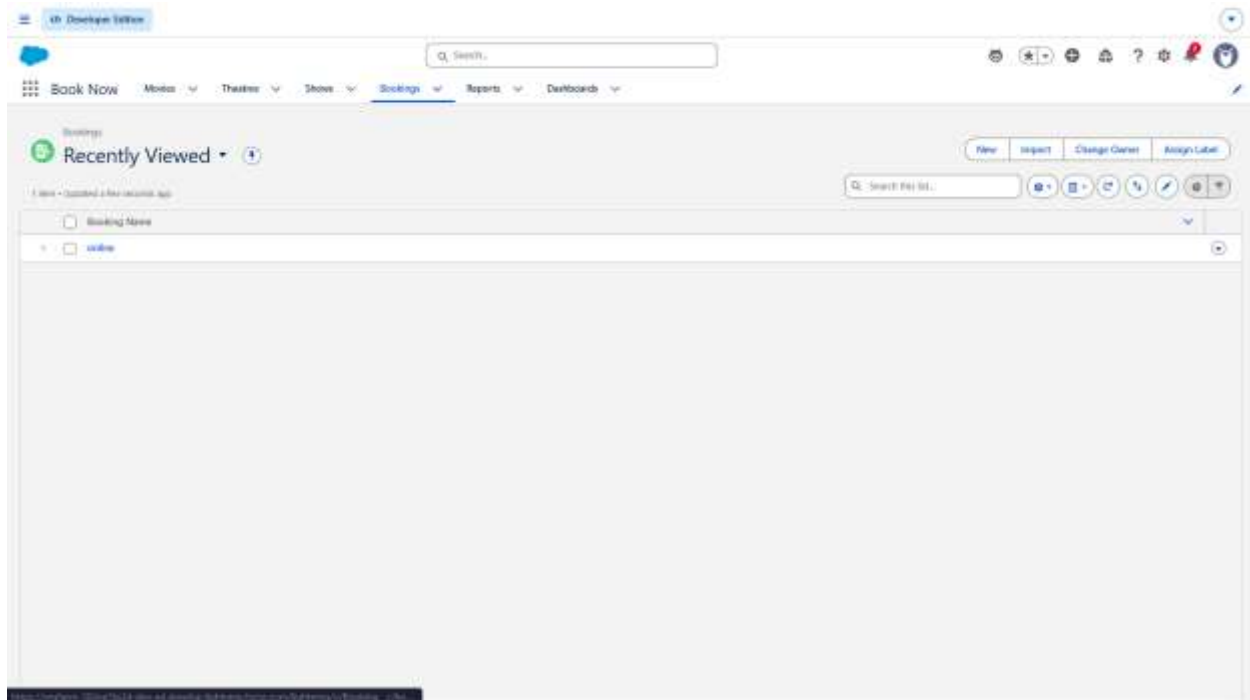


Figure 2.27 – Booking Tab



Implementation Outcome

- Improved application navigation.
- Faster access to business records.
- Better user experience.
- Organized Lightning Application interface.
- Simplified day-to-day record management.

2.9 Validation Rules

Introduction

Validation Rules were implemented to ensure that only valid booking information is stored in the Book Now Movie Booking Application. By enforcing business rules before records are saved,

Salesforce helps maintain data accuracy and prevents invalid bookings from affecting seat availability and reporting.

Business Rule

The application does not allow users to create a booking with **zero or negative seats**. If an invalid value is entered, Salesforce prevents the record from being saved and displays an appropriate error message.

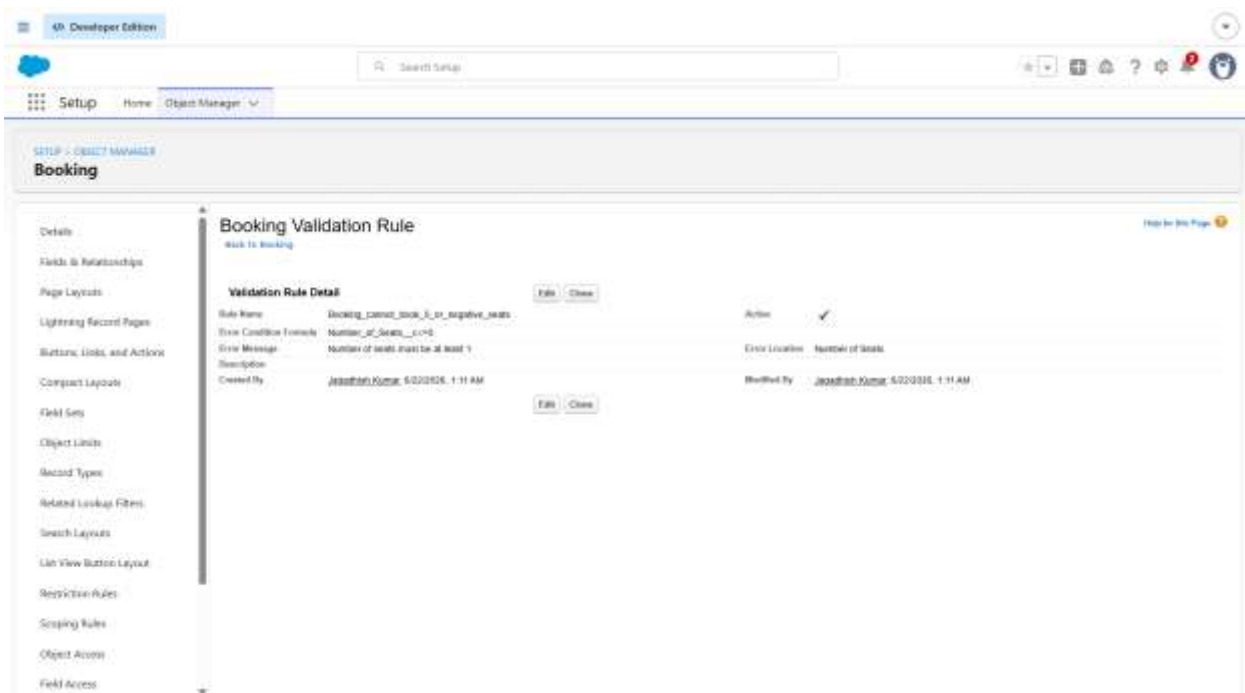
Validation Formula

Number_of_Seats__c <= 0

Error Message

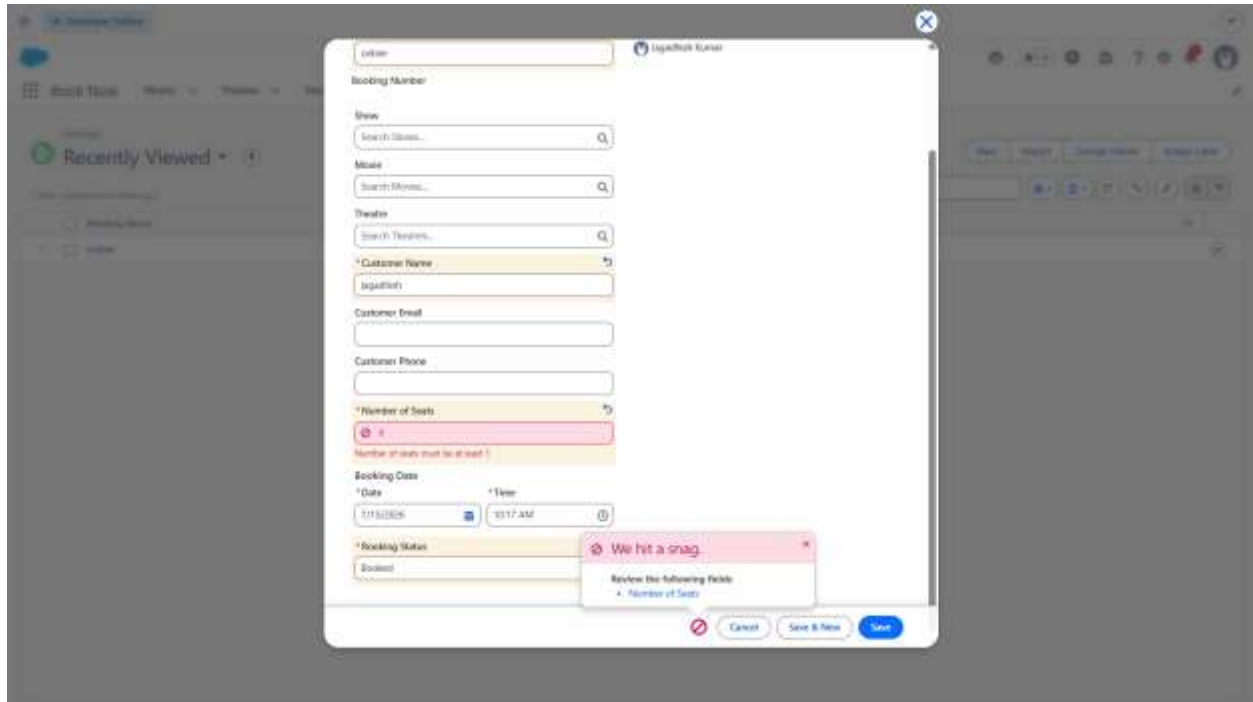
"Number of seats must be at least 1 "

Figure 2.28 – Validation Rule Configuration



Screenshot Description: Displays the Validation Rule configuration, including the formula and error message.

Figure 2.29 – Validation Rule Testing



Screenshot Description: Displays the validation error when a booking is created with an invalid seat count.

Implementation Outcome

- Prevents invalid booking records.
 - Improves data accuracy.
 - Enforces business rules consistently.
 - Reduces manual verification.
 - Improves the reliability of reports and seat management.
-

2.10 Formula Fields

Introduction

Formula Fields were implemented to automate calculations within the Book Now Movie Booking Application. Instead of manually calculating the total booking amount, Salesforce automatically computes the value based on the ticket price and the number of seats booked. This improves accuracy, reduces manual effort, and ensures consistent pricing across all booking records.

Business Rule

The total booking amount is calculated automatically using the following formula:

Total Amount = Ticket Price × Number of Seats

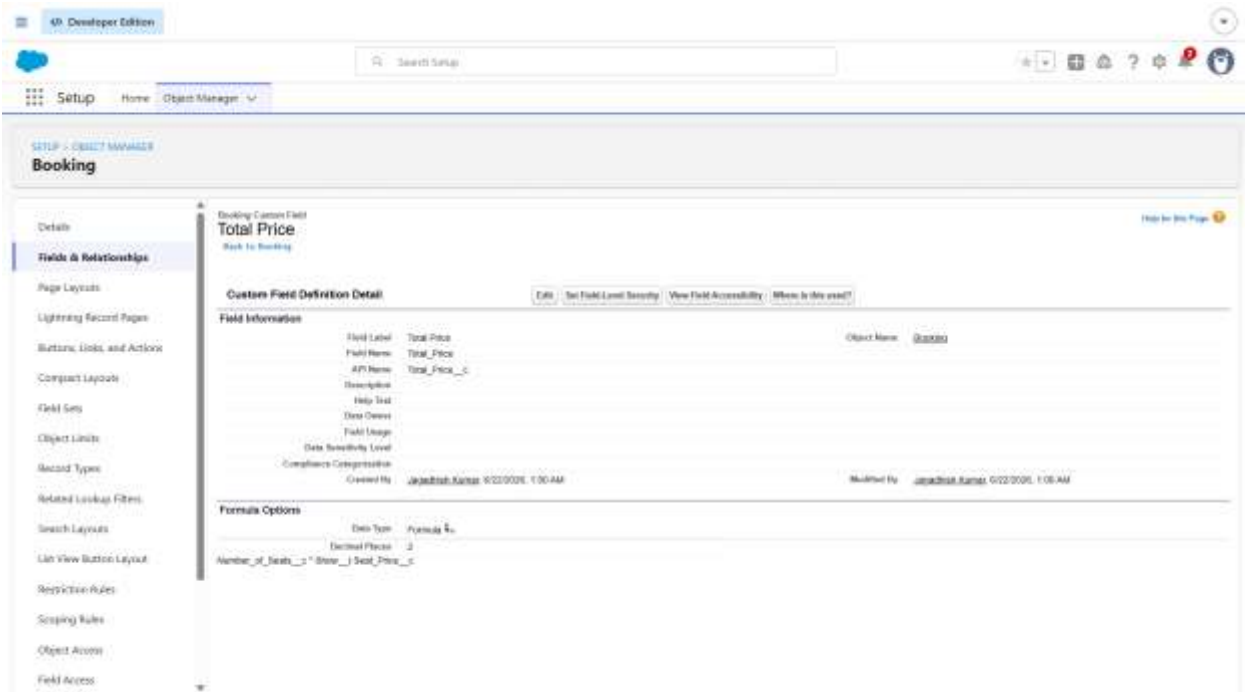
This calculation is performed whenever a booking record is created or updated.

Formula Expression

Show__r.Ticket_Price__c * Number_of_Seats__c

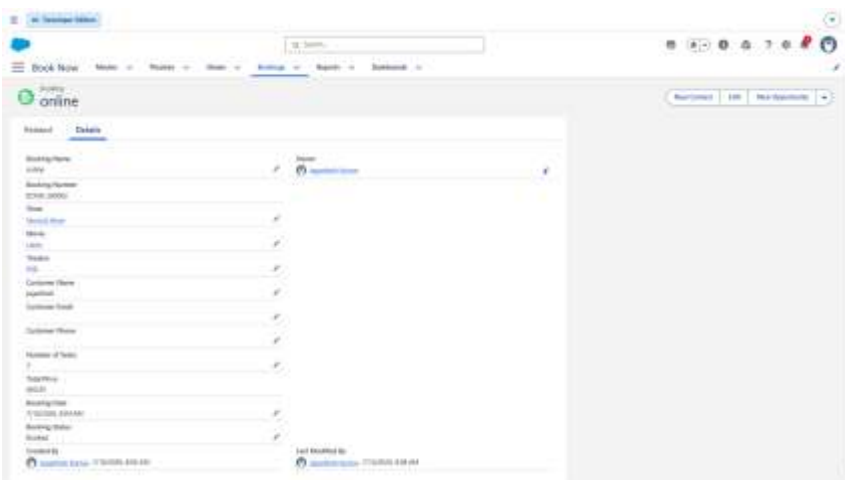
Replace this with the exact formula from your Salesforce org if it differs.

Figure 2.30 – Formula Field Configuration



Screenshot Description: Displays the Formula Field configuration.

Figure 2.31 – Formula Field Result



Screenshot Description: Displays a booking record showing the automatically calculated Total Amount.

Implementation Outcome

- Eliminates manual calculations.
 - Reduces human errors.
 - Ensures accurate ticket pricing.
 - Simplifies booking management.
 - Improves reporting accuracy.
-

2.11 Apex Trigger Implementation

Introduction

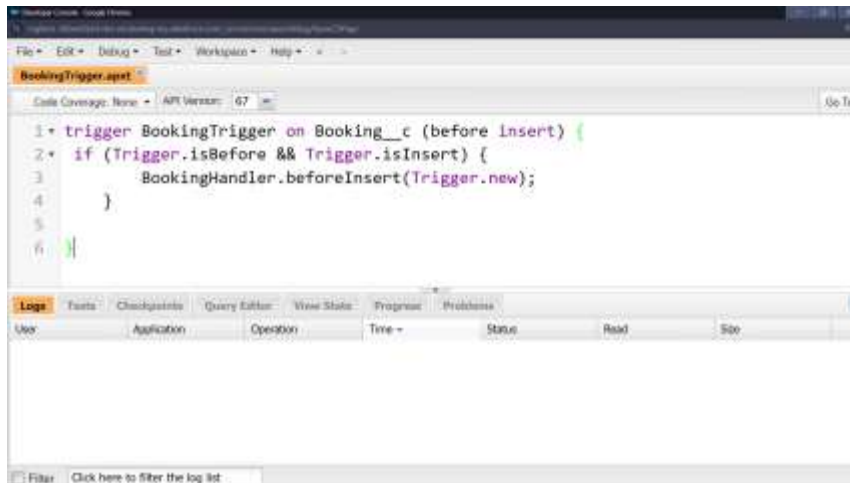
An **Apex Trigger** was implemented to automate seat availability management in the Book Now Movie Booking Application. Whenever a new booking is created, the trigger executes automatically and invokes the **BookingTriggerHandler** class to update the available seats for the selected show. This automation ensures that booking information and seat availability remain synchronized without manual intervention.

Trigger Functionality

The Apex Trigger performs the following operations:

- Executes automatically after a booking record is created.
 - Passes booking records to the **BookingTriggerHandler** class.
 - Retrieves the corresponding Show record.
 - Updates the available seat count.
 - Saves the updated Show record.
-

Figure 2.32 – Apex Trigger

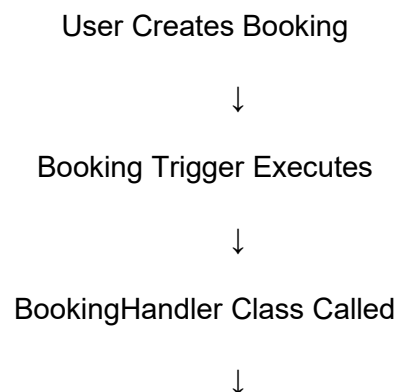


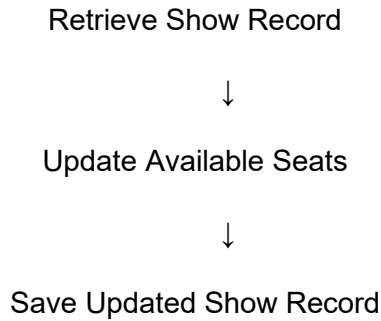
Screenshot Description: Displays the Apex Trigger (**BookingTrigger**) in the Developer Console or VS Code.

Trigger Execution Flow

1. A new booking is created.
2. The Apex Trigger executes automatically.
3. Control is passed to the **BookingTriggerHandler** class.
4. The handler retrieves the related Show record.
5. Available seats are reduced based on the booking.
6. The updated Show record is saved.

Figure 2.33 – Trigger Execution Flow





Implementation Outcome

- Automated seat availability updates.
 - Eliminated manual seat management.
 - Improved booking accuracy.
 - Maintained data consistency.
 - Followed Salesforce best practices using a separate handler class.
-

2.12 Apex Handler Class

Introduction

The **BookingTriggerHandler** class contains the business logic responsible for updating seat availability after a booking is created. Separating the business logic from the Apex Trigger follows Salesforce best practices, making the application easier to maintain, test, and extend.

Handler Functionality

The **BookingTriggerHandler** performs the following tasks:

- Receives booking records from the Apex Trigger.
- Retrieves the related Show records.
- Calculates the remaining available seats.
- Updates the Show records with the new seat count.
- Saves the changes to the database.

Processing Flow

1. Receive booking records from the trigger.
 2. Retrieve the related Show records.
 3. Calculate the remaining available seats.
 4. Update the Show records.
 5. Save the updated records.
-

Figure 2.34 – BookingTriggerHandler Class

```
public class BookingHandler {  
  
    public static void beforeInsert(List<Booking__c> newList){  
  
        Set<Id> showIds = new Set<Id>();  
  
        for(Booking__c b : newList){  
  
            if(b.Show__c != null) showIds.add(b.Show__c);  
  
            if(b.No_of_Seats__c == null) b.No_of_Seats__c = 1;  
  
        }  
  
        if(showIds.isEmpty()) return;  
  
  
        // Lock the show records to prevent race conditions  
  
        Map<Id, Show__c> shows = new Map<Id, Show__c>([SELECT  
Id, Available_Seats__c FROM Show__c WHERE Id IN :showIds FOR  
UPDATE]);  
  
  
        // Aggregate requested seats per show  
  
        Map<Id, Integer> seatsRequested = new Map<Id,  
Integer>();
```

```

        for(Booking__c b : newList){

            if(b.Show__c == null) continue;

            Integer req = (b.No_of_Seats__c == null) ? 0 :
Integer.valueOf(b.No_of_Seats__c);

            seatsRequested.put(b.Show__c,
seatsRequested.containsKey(b.Show__c) ?
seatsRequested.get(b.Show__c) + req : req);

        }

        List<Show__c> showsToUpdate = new List<Show__c>();

        // Validate and reduce available seats

        for(Id sid : seatsRequested.keySet()){

            Show__c s = shows.get(sid);

            Integer req = seatsRequested.get(sid);

            Integer avail = (s.Available_Seats__c == null) ? 0 :
Integer.valueOf(s.Available_Seats__c);

            if(req > avail){

                // Add error to any booking for this show

                for(Booking__c b : newList){

                    if(b.Show__c == sid){

                        b.addError('Not enough seats available.
Available: ' + avail);

                    }

                }

            } else {

                s.Available_Seats__c = avail - req;

```

```
showsToUpdate.add(s);  
  
}  
  
}  
  
if(!showsToUpdate.isEmpty()){  
    update showsToUpdate;  
}  
  
}  
  
}
```

Screenshot Description: Displays the `BookingTriggerHandler` Apex class.

Implementation Outcome

- Centralized business logic.
 - Automatic seat availability updates.
 - Support for bulk record processing.
 - Reduced code duplication.
 - Improved maintainability and scalability.
-

2.13 Apex Test Class

Introduction

An **Apex Test Class** was developed to verify the functionality of the **BookingTrigger** and **BookingTriggerHandler**. The test class validates the booking process, seat availability updates, and ensures that the Apex code meets Salesforce deployment requirements.

Test Scenarios

Test Scenario	Expected Result
Create a valid booking	Booking record is created successfully.
Update available seats	Available seats are reduced correctly.
Invalid seat booking	Validation Rule prevents record creation.
Trigger execution	BookingTrigger invokes BookingTriggerHandler successfully.

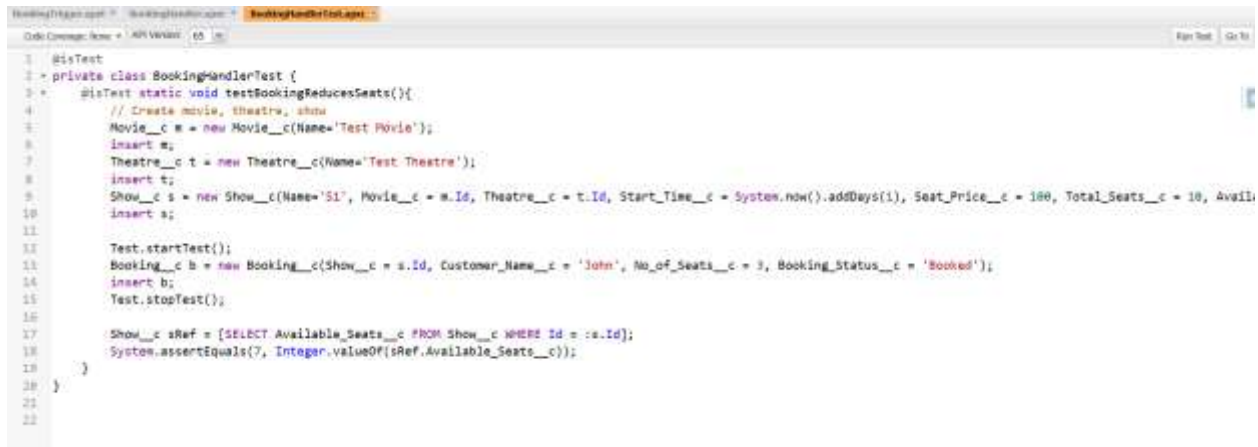
Code Coverage

The implemented test class achieved the minimum code coverage required for Salesforce deployment.

Actual Project Result:

- **Overall Code Coverage: 82%**
 - **BookingTrigger: 100%**
 - **BookingTriggerHandler: 81%**
-

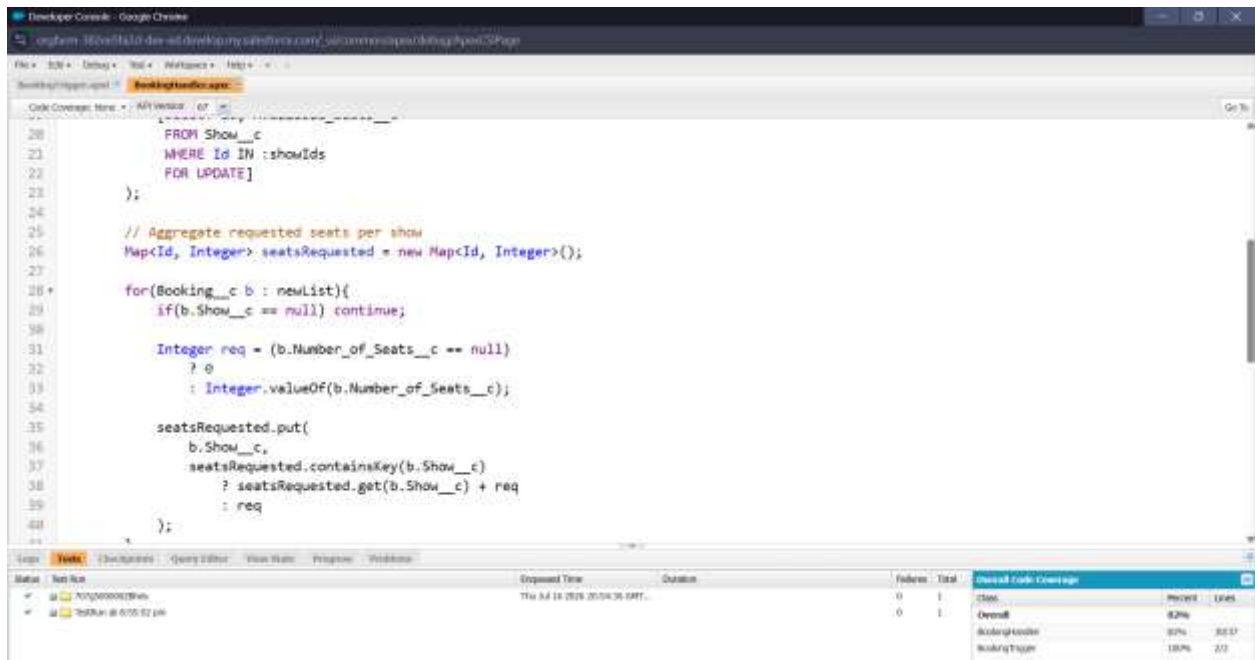
Figure 2.35 – Apex Test Class



```
1  @isTest
2  private class BookingHandlerTest {
3  *   @isTest static void testBookingReducesSeats(){
4      // Create movie, theatre, show
5      Movie__c m = new Movie__c(Name='Test Movie');
6      insert m;
7      Theatre__c t = new Theatre__c(Name='Test Theatre');
8      insert t;
9      Show__c s = new Show__c(Name='S1', Movie__c = m.Id, Theatre__c = t.Id, Start_Time__c = System.now().addDays(1), Seat_Price__c = 100, Total_Seats__c = 10, Avail:
10     insert s;
11
12     Test.startTest();
13     Booking__c b = new Booking__c(Show__c = s.Id, Customer_Name__c = 'John', No_of_Seats__c = 3, Booking_Status__c = 'Booked');
14     insert b;
15     Test.stopTest();
16
17     Show__c sRef = [SELECT Available_Seats__c FROM Show__c WHERE Id = :s.Id];
18     System.assertEquals(7, Integer.valueOf(sRef.Available_Seats__c));
19 }
20 }
21
22
```

Screenshot Description: Displays the `BookingHandlerTest` Apex test class.

Figure 2.36 – Test Execution Results



The screenshot shows the Salesforce IDE interface. The top pane displays the source code of the `BookingHandlerTest` class, with the `testBookingReducesSeats` method highlighted. The bottom pane shows the test execution results. The test run was successful, with a status of 'Succeeded' and a duration of 0:00:00. The overall code coverage is 82%, with 82% for the `BookingHandler` class and 100% for the `BookingTrigger` class.

Test Run	Elapsed Time	Duration	Passes	Total	Overall Code Coverage
TestRun at 5:05:52 pm	Thu Jul 19 2019 20:04:36 GMT...		0	1	
			0	1	
					Overall
					BookingHandler
					BookingTrigger

Screenshot Description: Displays the successful execution of Apex tests and the code coverage results.

Implementation Outcome

- Verified trigger functionality.
 - Validated seat availability updates.
 - Confirmed successful booking creation.
 - Achieved the required Salesforce code coverage.
 - Improved application reliability through automated testing.
-

2.14 Agentforce Configuration & AI Integration

Introduction

To enhance the user experience, the Book Now Movie Booking Application integrates **Salesforce Agentforce**, an AI-powered conversational assistant that enables users to retrieve movie information using natural language. Instead of manually searching records, users can ask questions about movies and receive instant responses based on data stored in Salesforce.

The Agentforce implementation demonstrates how Artificial Intelligence can improve CRM applications by providing intelligent, real-time access to business information while reducing manual effort.

Business Requirement

The application required an AI assistant capable of answering common movie-related questions without requiring users to manually browse Salesforce records.

The solution allows users to retrieve information such as:

- Movie details
- Movie language
- Duration
- Release date
- Movie description

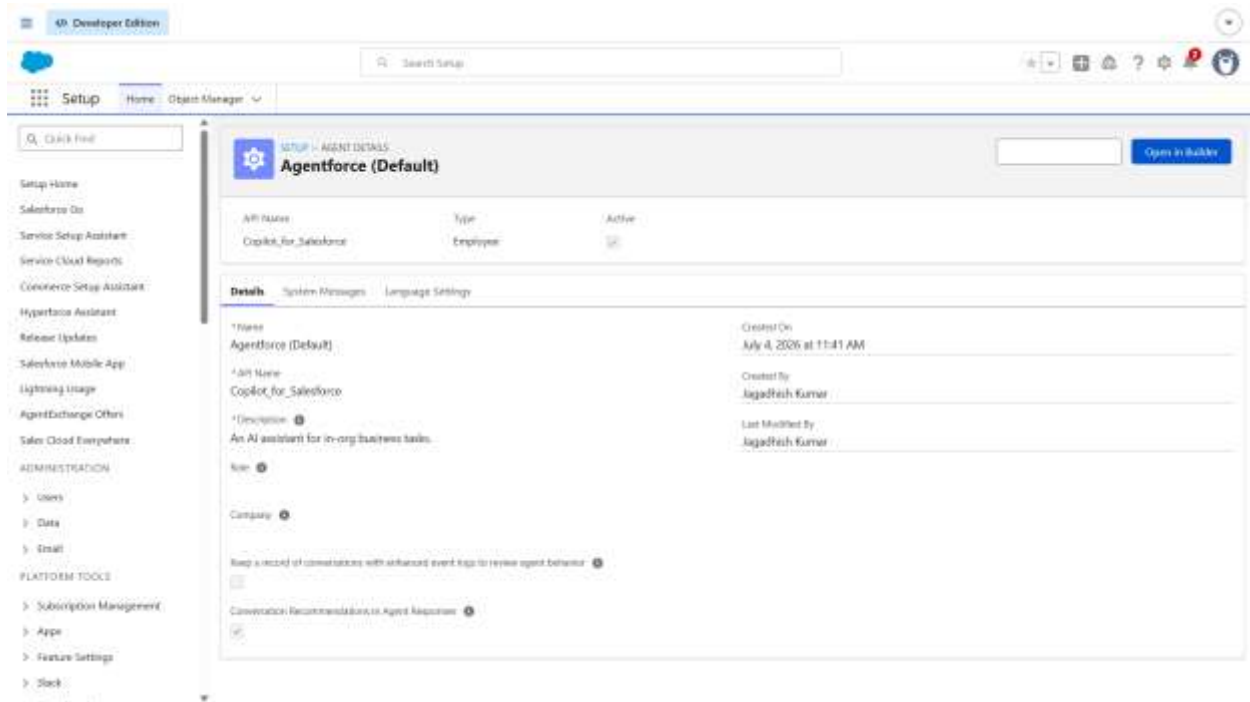
This functionality improves response time and enhances the overall customer experience.

Agent Configuration

The Agentforce agent was configured using Salesforce Agent Builder. The configuration included:

- Creating the Agent.
- Defining the agent's purpose.
- Configuring system instructions.
- Creating Topics.
- Linking Actions.
- Publishing the agent for testing.

Figure 2.37 – Agent Configuration



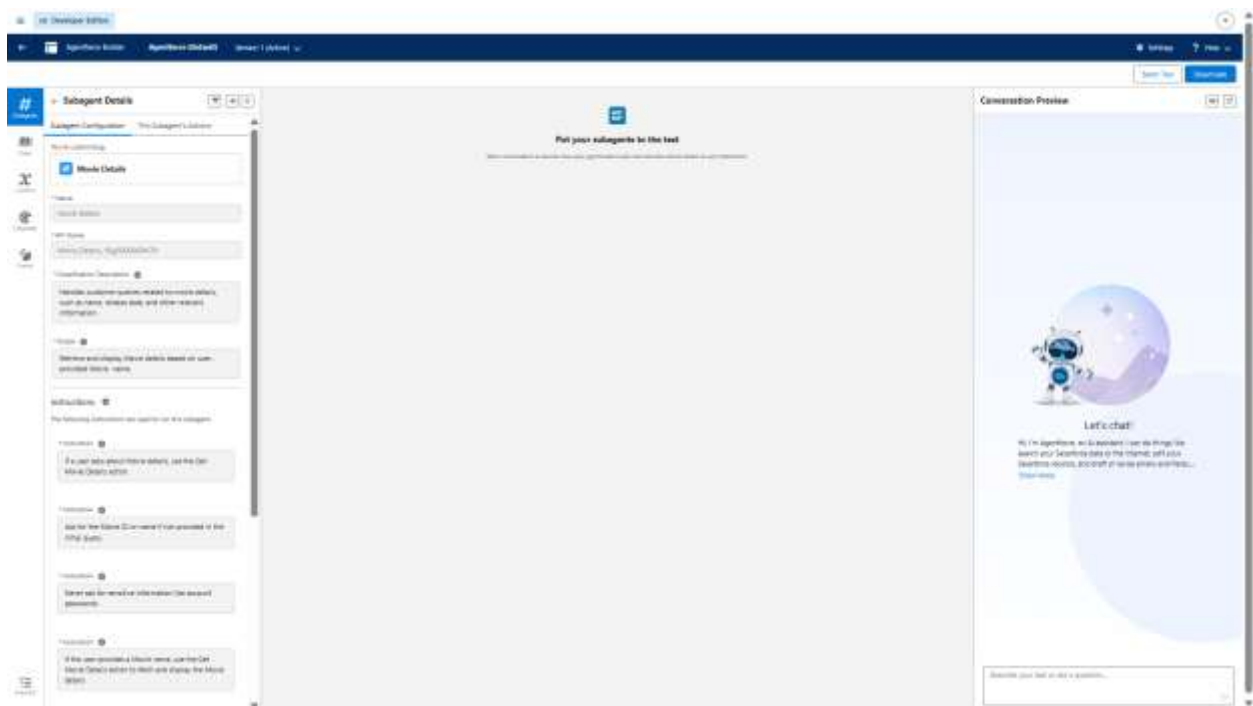
Agent Topics

Topics define the types of conversations the AI assistant can understand.

The following topic was configured for this project:

Topic	Purpose
Movie Details	Retrieves movie information from Salesforce records.

Figure 2.38 – Agent Topics



Agent Actions

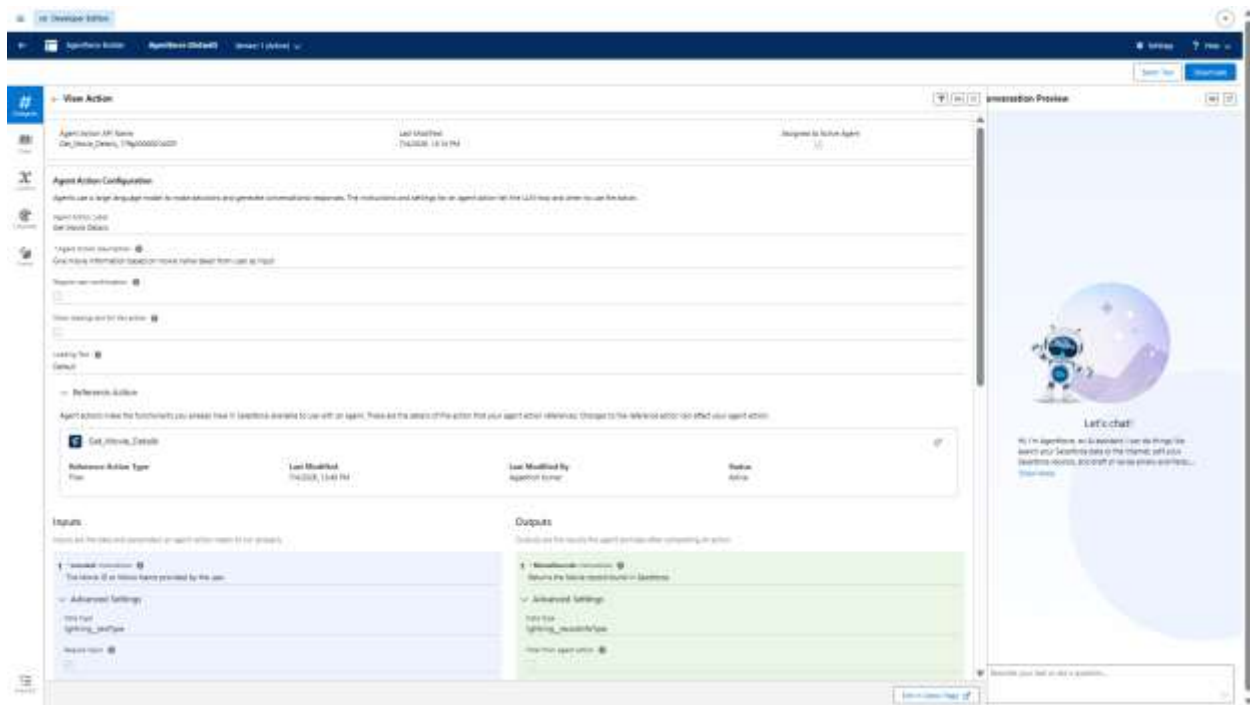
Agent Actions determine what Agentforce performs after identifying the user's request.

For this project, the configured action:

- Accepts the movie name.

- Invokes the Salesforce Flow.
- Retrieves movie information.
- Returns the response to the user.

Figure 2.39 – Agent Actions



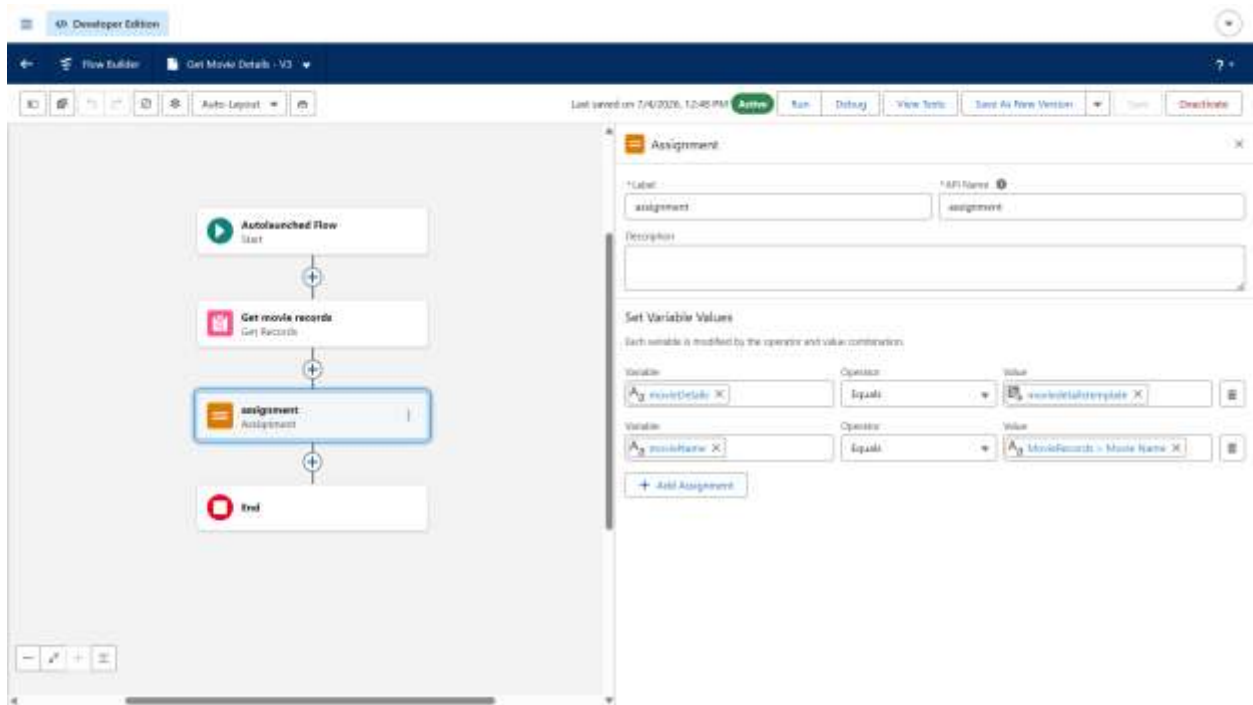
Flow Integration

Agentforce communicates with Salesforce through a Flow named **Get_Movie_Details**.

The Flow performs the following steps:

1. Accepts the movie name from Agentforce.
2. Searches the **Movie** object.
3. Retrieves the matching movie record.
4. Returns the requested information.
5. Sends the response back to Agentforce.

Figure 2.40 – Agentforce Flow



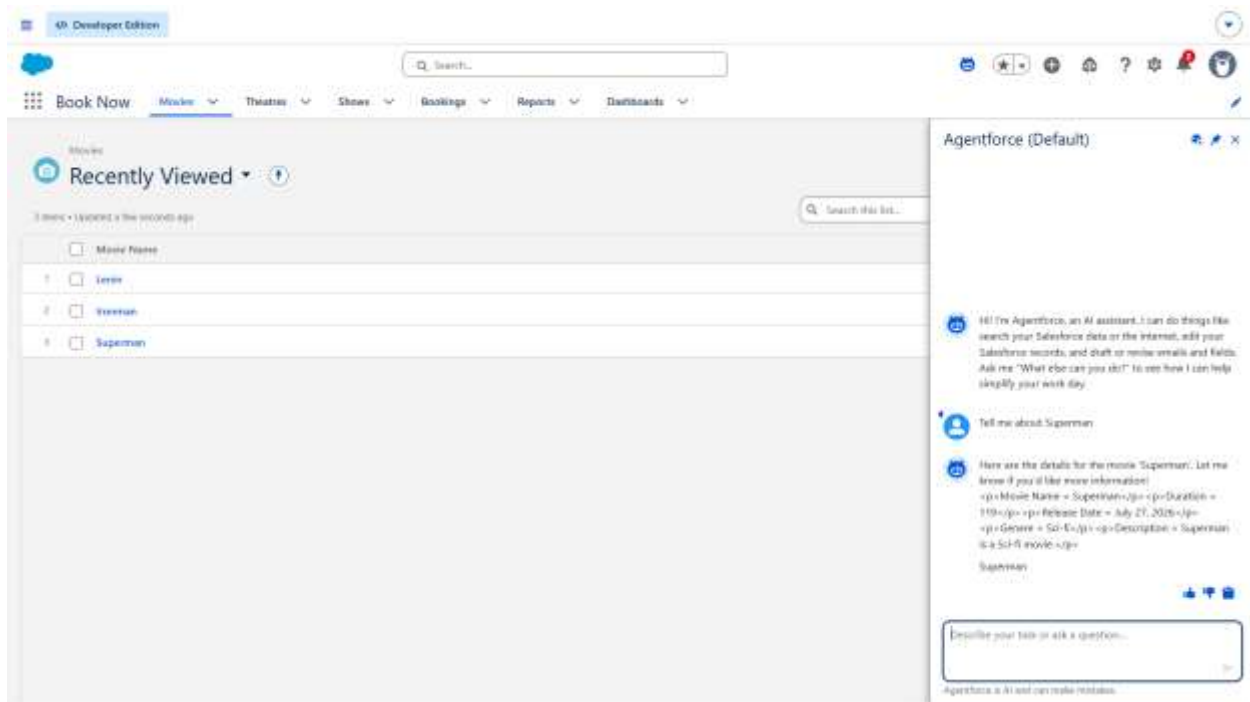
Conversation Testing

The Agentforce implementation was tested using different movie-related queries to verify that the configured Topic, Action, and Flow worked correctly.

User Query	Expected Response
Tell me about Kalki	Displays movie details.
What is the duration of Pushpa?	Returns movie duration.
Which language is Leo available in?	Displays movie language.

Give me the description of Salaar.	Returns movie description.
------------------------------------	----------------------------

Figure 2.41 – Agentforce Conversation



Implementation Outcome

The integration of Agentforce provides several benefits:

- Faster retrieval of movie information.
- Improved customer experience.
- Reduced manual effort.
- Accurate responses from Salesforce records.
- Demonstrates Salesforce AI capabilities.
- Provides a scalable foundation for future AI enhancements.

2.15 Chapter Summary

This chapter presented the backend development of the Book Now Movie Booking Application using Salesforce. It covered the implementation of custom objects, object relationships, custom fields, tabs, validation rules, formula fields, Apex Trigger, BookingTriggerHandler, Apex Test Class, and Agentforce integration.

By combining Salesforce declarative features with Apex programming, the application automates the movie booking process, maintains data integrity, and provides an AI-powered user experience through Agentforce. The completed backend establishes a scalable and maintainable foundation for the overall application.

CHAPTER 3

UI/UX Development & Customization

3.1 Introduction

The user interface plays an important role in ensuring that users can efficiently interact with the Book Now Movie Booking Application. Salesforce Lightning Experience was used to develop a clean, responsive, and user-friendly interface that simplifies navigation and improves productivity.

This chapter focuses on the customization of the Lightning Application, navigation, page layouts, record pages, compact layouts, and list views to provide an organized experience for administrators managing movies, theatres, shows, and customer bookings.

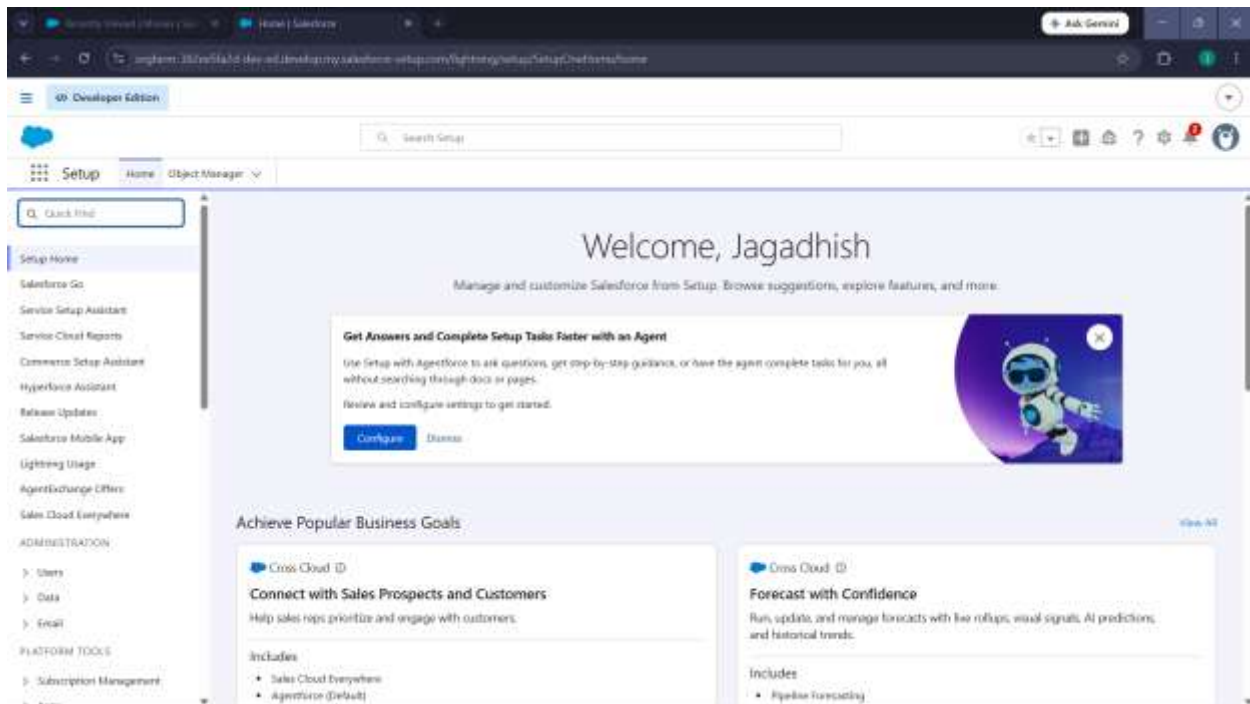
3.2 Objectives of UI/UX Development

The objectives of this phase were to:

- Develop a user-friendly Lightning Application.
- Simplify navigation between business objects.
- Customize page layouts and record pages.

- Improve productivity through organized user interfaces.
- Provide a consistent user experience.
- Support future customization and scalability.

Figure 3.1 – Salesforce Lightning Experience



3.3 Lightning Application Development

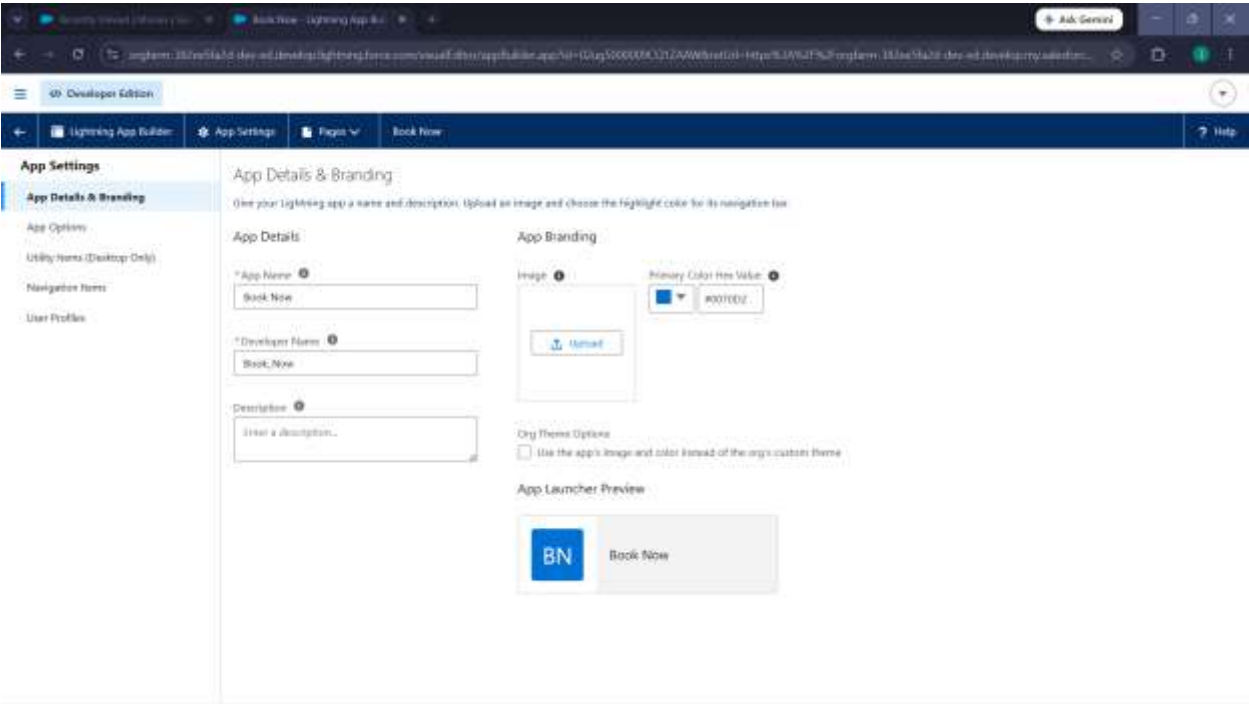
Introduction

A custom **Lightning Application** named **Book Now** was created using Salesforce App Manager to provide a centralized workspace for managing all movie booking operations. The application combines the required custom objects, reports, dashboards, and Agentforce into a single interface, allowing administrators to perform daily tasks efficiently.

Lightning Application Configuration

Configuration	Value
App Name	Book Now
Navigation Style	Standard Navigation
Tabs Included	Movie, Theatre, Show, Booking, Reports, Dashboards
Supported Platform	Salesforce Lightning Experience

Figure 3.2 – Lightning Application Configuration

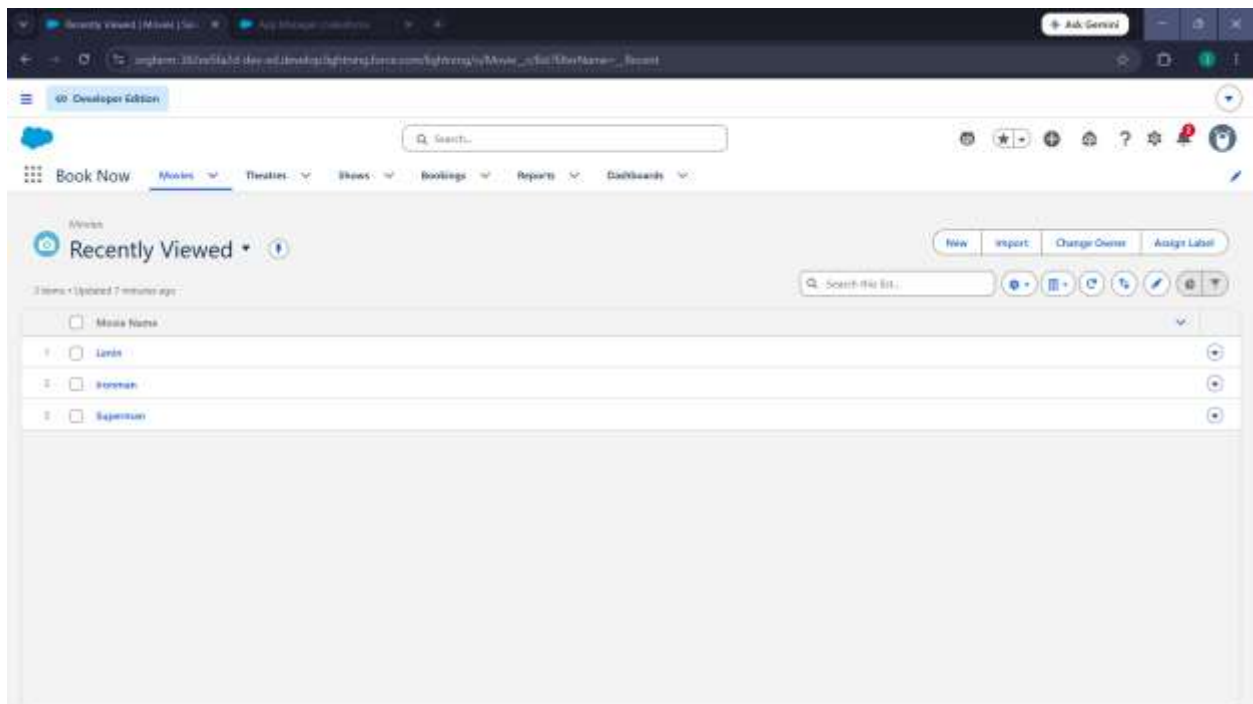


Implementation Outcome

The custom Lightning Application provides:

- Centralized navigation.
- Faster access to application records.
- Improved user experience.
- Better productivity.
- Easy scalability for future enhancements.

Figure 3.3 – Book Now Lightning Application



3.4 Navigation Menu Design

Introduction

The navigation menu was customized to provide quick access to the most frequently used features of the Book Now Movie Booking Application. The tabs were arranged according to the typical workflow followed by administrators, making navigation simple and efficient.

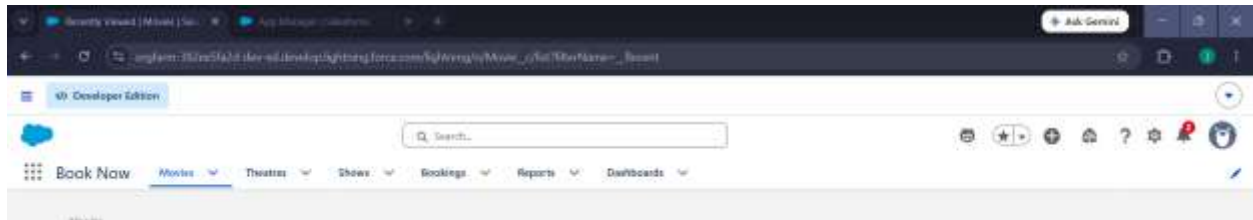
Navigation Order

- Home
- Movie
- Theatre
- Show
- Booking
- Reports
- Dashboards
- Agentforce (if configured)

Implementation Outcome

- Faster navigation between objects.
- Improved workflow efficiency.
- Better user experience.
- Consistent application layout.
- Reduced time required to access records.

Figure 3.4 – Navigation Menu



3.5 Page Layout Customization

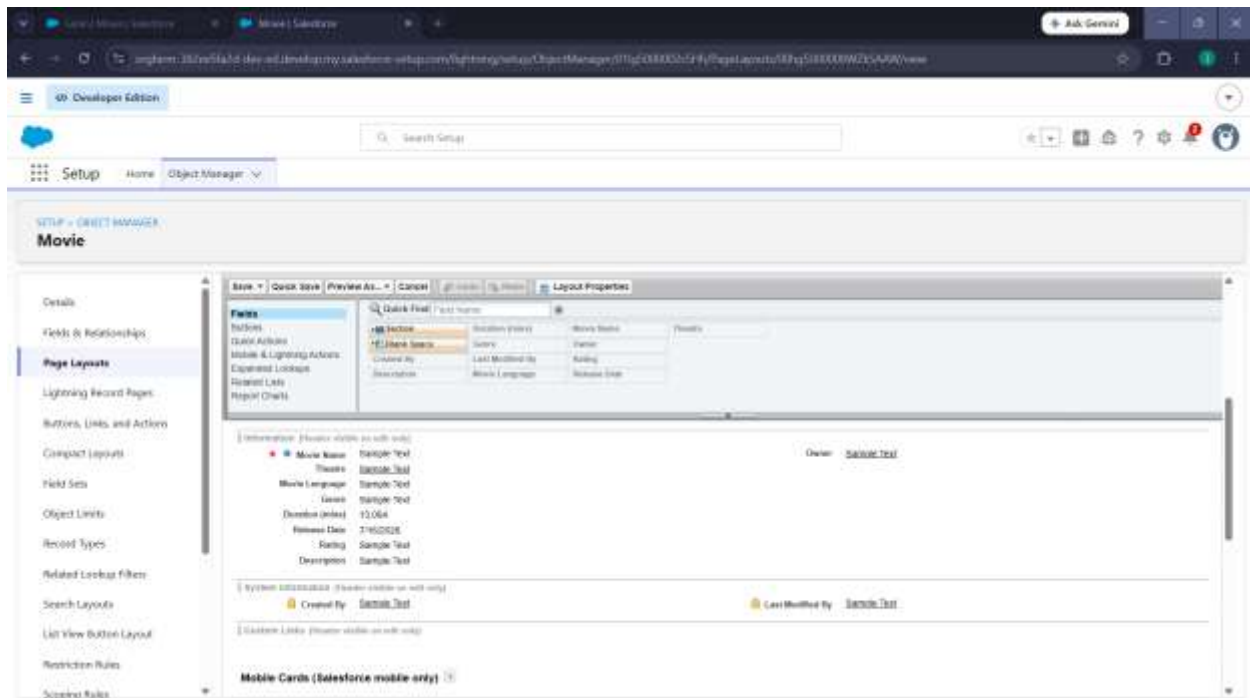
Introduction

Custom page layouts were created for each object to organize information in a clear and user-friendly manner. Related fields were grouped together, making it easier for users to create, view, and update records while improving the overall usability of the application.

Movie Page Layout

The Movie page layout displays movie information such as title, genre, language, duration, release date, rating, and description in an organized format, enabling administrators to efficiently manage movie records.

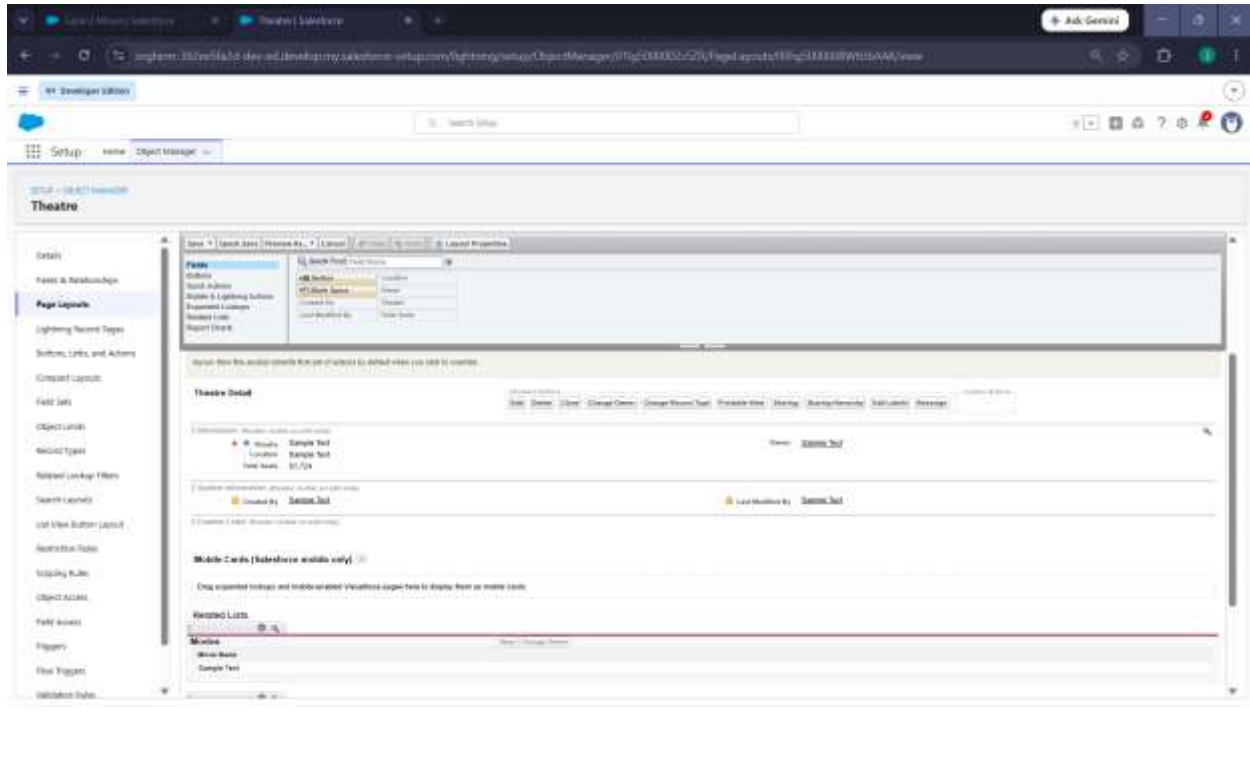
Figure 3.5 – Movie Page Layout



Theatre Page Layout

The Theatre page layout organizes theatre details such as theatre name, location, seating capacity, and status, allowing administrators to manage theatre records efficiently.

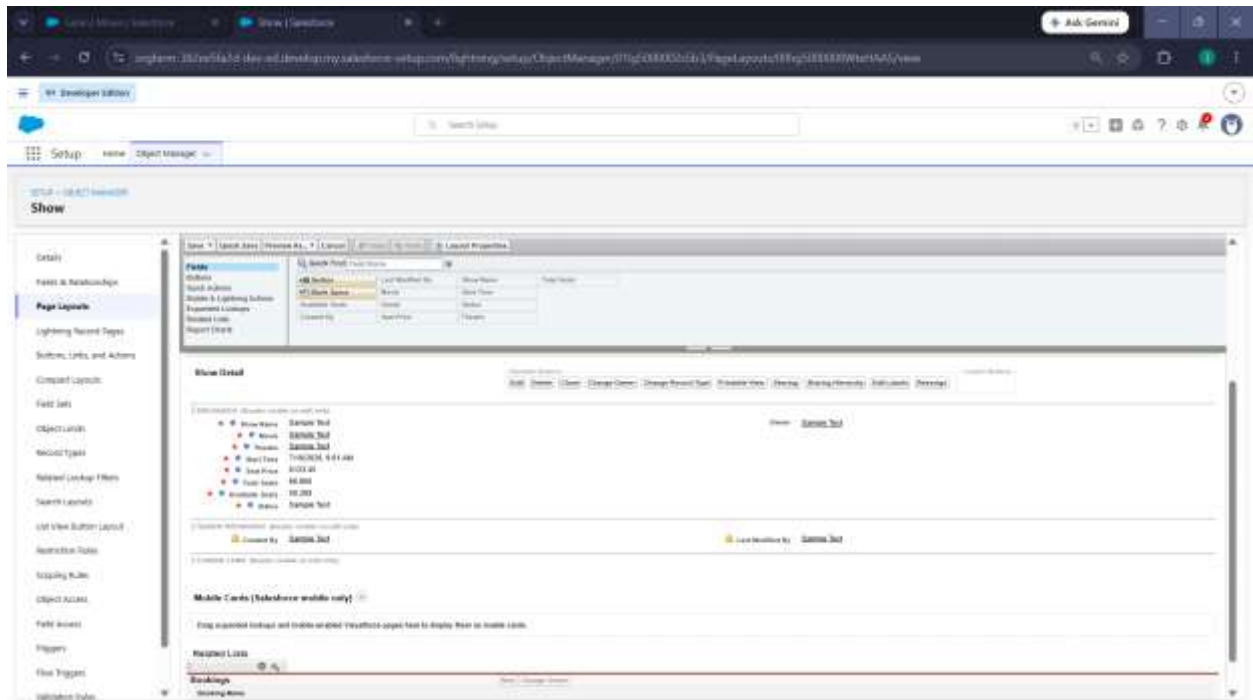
Figure 3.6 – Theatre Page Layout



Show Page Layout

The Show page layout displays all information required to manage movie screenings, including the associated movie, theatre, show date, show time, ticket price, total seats, and available seats. This organized layout enables administrators to efficiently schedule and manage movie shows.

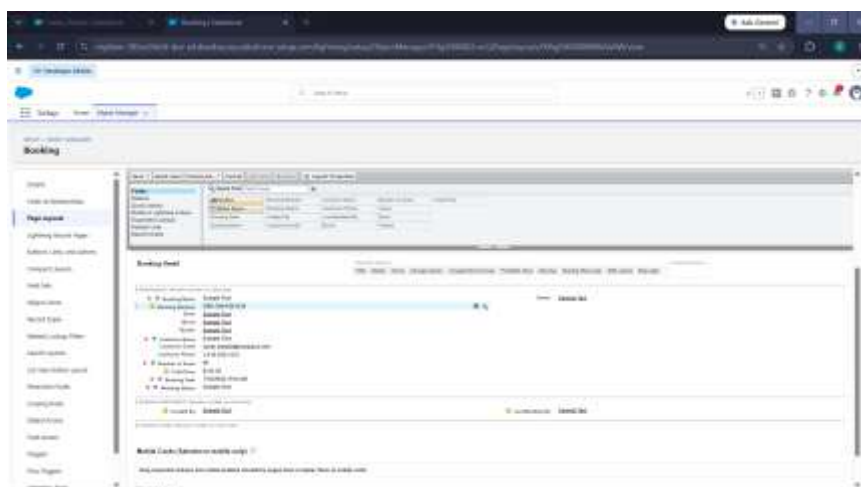
Figure 3.7 – Show Page Layout



Booking Page Layout

The Booking page layout presents customer details, booking information, selected show, number of seats, and the automatically calculated booking amount. This layout simplifies reservation management and allows administrators to quickly verify booking details.

Figure 3.8 – Booking Page Layout



Implementation Outcome

- Improved record organization.
- Faster record management.
- Reduced data entry errors.
- Better information visibility.
- Consistent user experience.

3.6 Lightning Record Pages

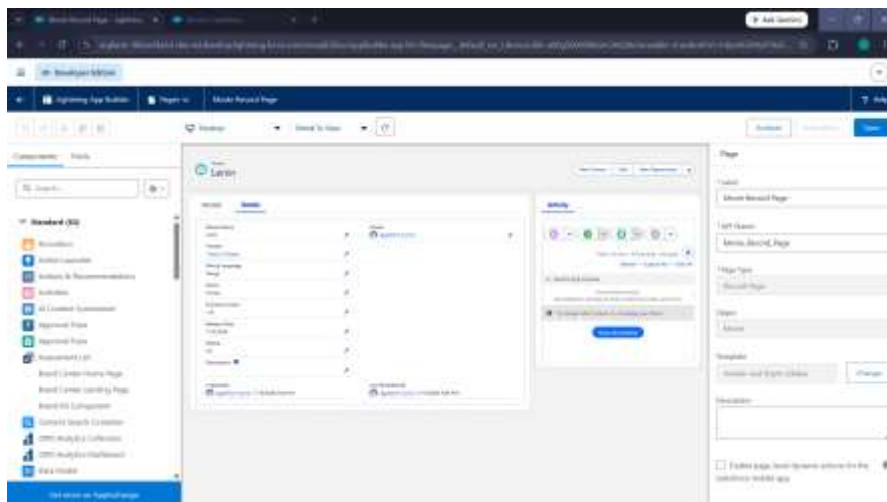
Introduction

Lightning Record Pages were customized using **Lightning App Builder** to provide an organized and responsive user interface. They display important record information, related records, and key actions, allowing users to manage data more efficiently.

Highlights Panel

The Highlights Panel displays important information at the top of each record, enabling users to quickly review essential details without opening multiple sections.

Figure 3.9 – Lightning Record Page



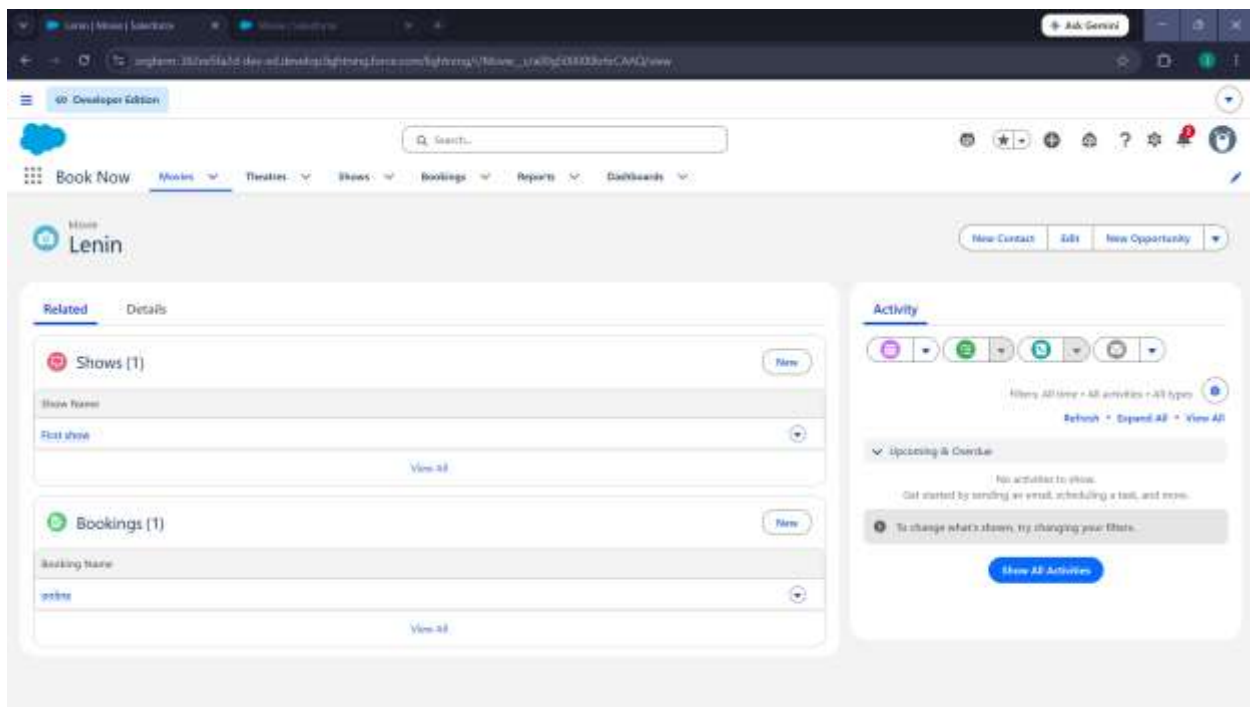
Related Lists

Related Lists display associated records directly within the parent record. For example:

- Shows related to a Movie.
- Shows associated with a Theatre.
- Bookings related to a Show.

This improves navigation and simplifies record management.

Figure 3.10 – Related Lists



Implementation Outcome

- Better visibility of important information.
- Faster navigation between related records.
- Improved productivity.
- Simplified record management.
- Responsive user experience.

3.7 Compact Layouts

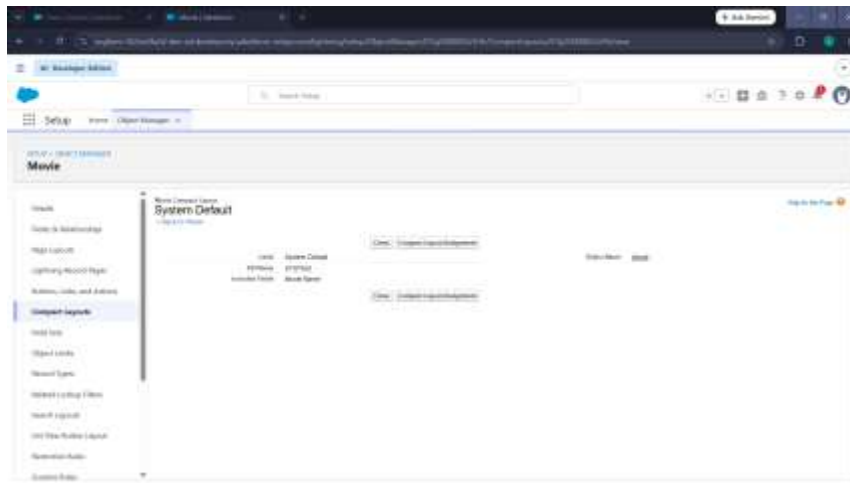
Introduction

Compact Layouts were configured to display important fields in the **Highlights Panel**, allowing users to quickly view essential record information without opening the complete record. This improves usability and supports faster navigation within Salesforce Lightning Experience.

Configured Compact Layouts

Object	Displayed Fields
Movie	Movie Name, Genre, Language, Duration
Theatre	Theatre Name, Location, Seating Capacity
Show	Show Name, Movie, Show Time, Available Seats
Booking	Booking Name, Customer Name, Number of Seats, Total Amount

Figure 3.11 – Compact Layout Configuration



Implementation Outcome

- Quick access to important record information.
 - Reduced navigation time.
 - Improved user productivity.
 - Better user experience across Lightning Experience..
-

3.8 List Views

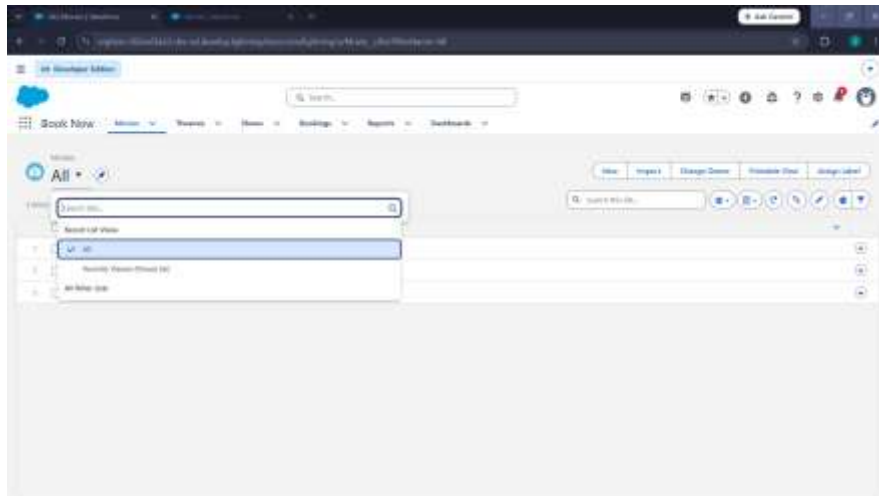
Introduction

List Views were configured to help users quickly access and manage records within the Book Now Movie Booking Application. They provide an organized view of records based on predefined criteria, reducing the time required to search for information.

Configured Compact Layouts

Object	Displayed Fields
Movie	Movie Name, Genre, Language, Duration
Theatre	Theatre Name, Location, Seating Capacity
Show	Show Name, Movie, Show Time, Available Seats
Booking	Booking Name, Customer Name, Number of Seats, Total Amount

Figure 3.12 – List View Configuration



Implementation Outcome

- Faster record retrieval.
- Simplified data management.
- Improved operational efficiency.

- Better organization of records.
 - Enhanced user productivity.
-

3.9 Reports

Introduction

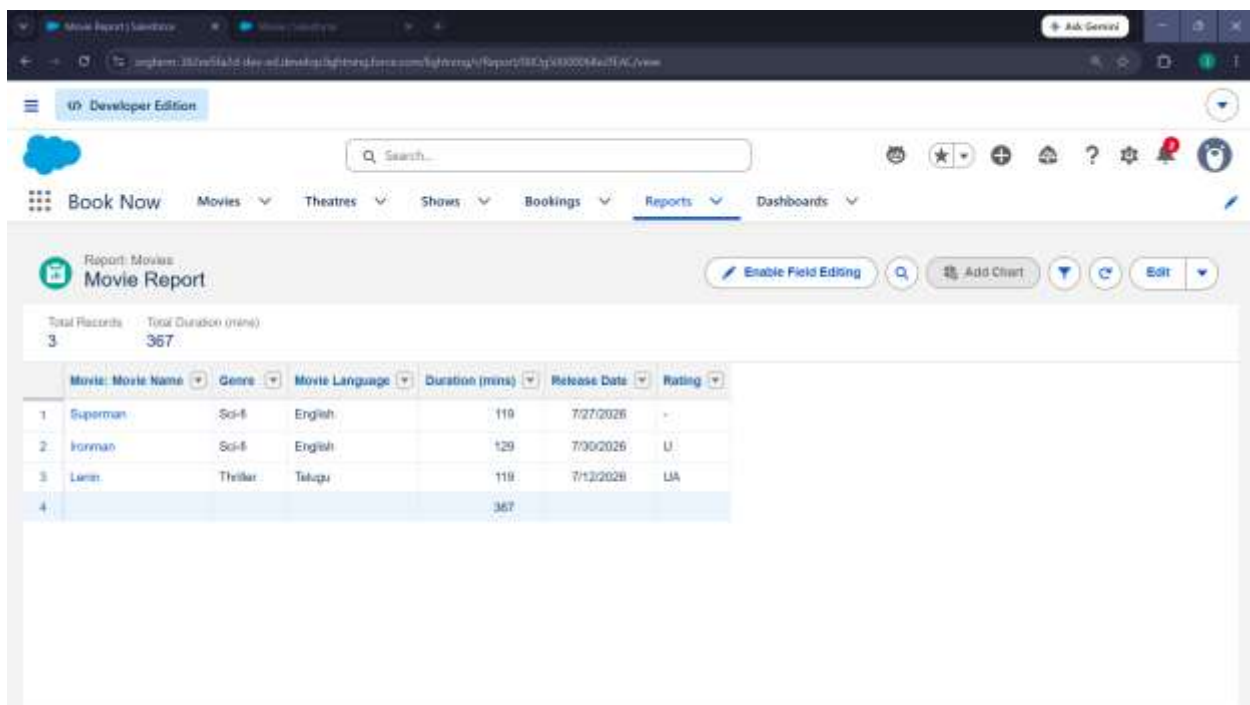
Salesforce Reports were created to help administrators monitor and analyze application data. They provide organized information about movies, theatres, shows, and bookings, supporting operational monitoring and decision-making.

Configured Reports

Report	Purpose
Movie Report	Displays movie details such as genre, language, duration, and release date.
Theatre Report	Displays theatre information including location and seating capacity.
Show Report	Displays scheduled shows, ticket prices, and seat availability.
Booking Report	Displays customer bookings, booked seats, and total booking amount.

Figure 3.13 – Reports

Movies Report

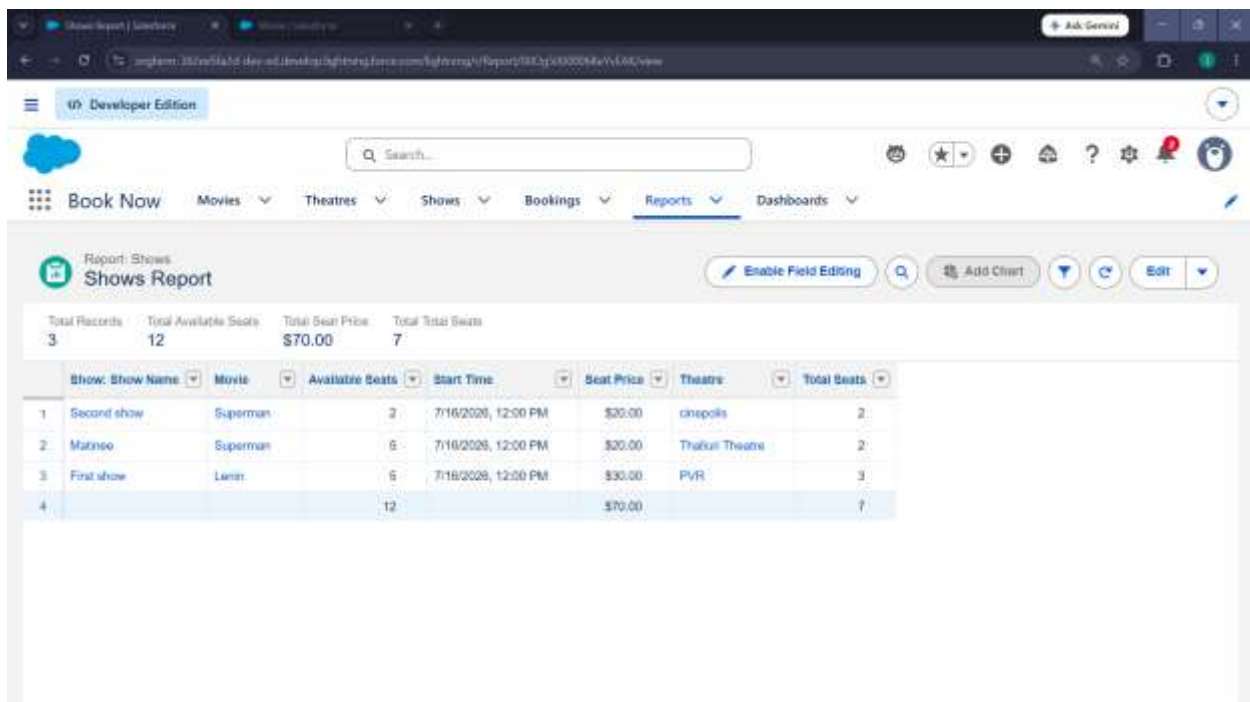


Report: Movies
Movie Report

Total Records: 3 Total Duration (mins): 367

	Movie: Movie Name	Genre	Movie Language	Duration (mins)	Release Date	Rating
1	Superman	Sci-Fi	English	119	7/27/2026	-
2	Ironman	Sci-Fi	English	129	7/30/2026	U
3	Lenin	Thriller	Telugu	119	7/12/2028	UA
4				367		

Shows Report



Report: Shows
Shows Report

Total Records: 3 Total Available Seats: 12 Total Seat Price: \$70.00 Total Total Seats: 7

	Show: Show Name	Movie	Available Seats	Start Time	Seat Price	Theatre	Total Seats
1	Second show	Superman	2	7/16/2026, 12:00 PM	\$20.00	cinepolis	2
2	Matinee	Superman	6	7/16/2026, 12:00 PM	\$20.00	Thakur Theatre	2
3	First show	Lenin	6	7/16/2026, 12:00 PM	\$30.00	PVR	3
4			12		\$70.00		7

Theatres Report

Implementation Outcome

- Faster access to business information.
 - Simplified operational monitoring.
 - Better reporting and analysis.
 - Improved decision-making.
-

3.10 Dashboards

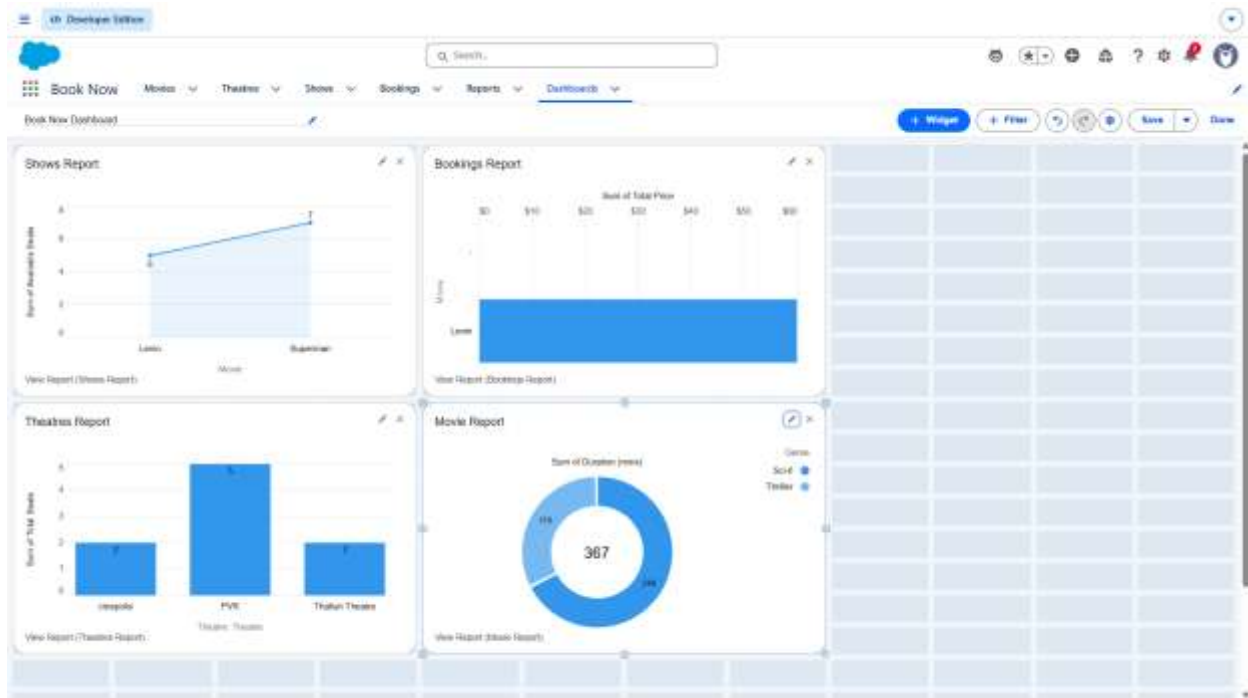
Introduction

Dashboards provide a graphical representation of report data using charts and summary components, allowing administrators to monitor business performance at a glance.

Dashboard Components

Component	Purpose
Movies by Genre	Displays movie distribution by genre.
Bookings by Show	Displays bookings for each show.
Theatre Capacity	Displays theatre seating capacity.
Booking Revenue	Displays total booking revenue.
Available Seats	Displays remaining seats for upcoming shows.

Figure 3.14 – Dashboard



Implementation Outcome

- Visual representation of data.
- Faster performance monitoring.
- Improved business insights.
- Better decision-making.

3.11 User Experience Evaluation

The application was tested to evaluate usability, navigation, consistency, and overall performance. The interface was found to be simple, responsive, and easy to use.

Evaluation Parameter	Observation
----------------------	-------------

Ease of Navigation	Simple and intuitive
--------------------	----------------------

Record Management	Efficient
Data Visibility	Clear and organized
Consistency	Uniform across all objects
Performance	Responsive
User Friendliness	Easy to learn

The evaluation confirmed that the application satisfies the project's usability requirements.

3.12 Future Enhancements

Future improvements may include:

- Lightning Web Components (LWC)
 - Dynamic Forms
 - Mobile optimization
 - AI recommendations
 - Interactive seat selection
 - Personalized home pages
 - Enhanced accessibility
-

3.13 Chapter Summary

This chapter explained the user interface development of the Book Now Movie Booking Application using Salesforce Lightning Experience. It covered the Lightning App, navigation, page layouts, record pages, compact layouts, list views, reports, dashboards, and usability

evaluation. The implemented interface provides an organized, user-friendly, and scalable environment for managing the movie booking process.

CHAPTER 4

Data Security, Migration & Testing

4.1 Introduction

This chapter explains the security configuration and testing performed for the Book Now Movie Booking Application. Salesforce security features were implemented to protect application data, while testing verified that all business processes function correctly before deployment.

4.2 Security Overview

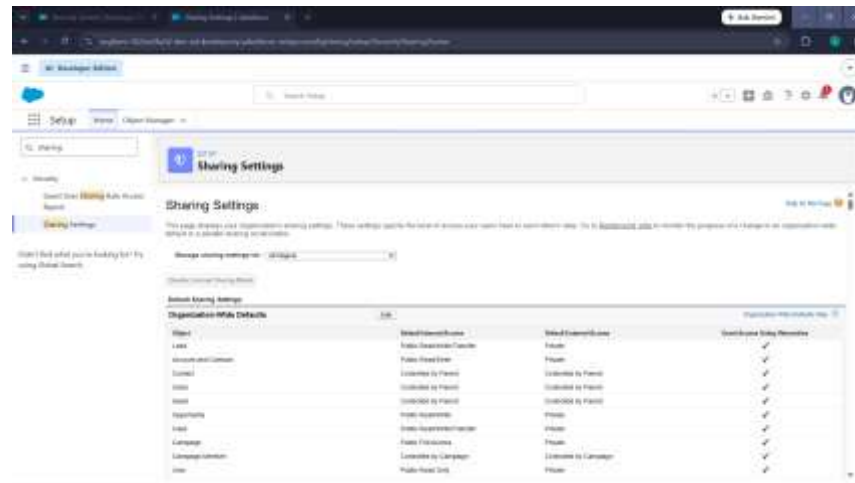
Introduction

Salesforce provides a layered security model to protect organizational data. The Book Now application uses Salesforce security features to ensure that users can access only the data and functionality required for their roles.

Security Features Implemented

- User Authentication
- Profiles
- Roles
- Object-Level Security
- Record-Level Security
- Field-Level Security (where applicable)

Figure 4.1 – Salesforce Security Configuration



4.3 Profiles

Introduction

Profiles define the permissions assigned to users. They control access to objects, fields, applications, tabs, and system features within Salesforce.

The project primarily used standard Salesforce profiles during development and testing.

Profile Permissions

Permission	Purpose
Read	View records
Create	Create records

Role Hierarchy

- System Administrator
- Theatre Manager
- Booking Executive
- Customer Support Executive

Figure 4.3 – Role Hierarchy

[illegible]

Implementation Outcome

- Organized role hierarchy.
- Simplified record sharing.
- Improved management oversight.
- Scalable security structure.

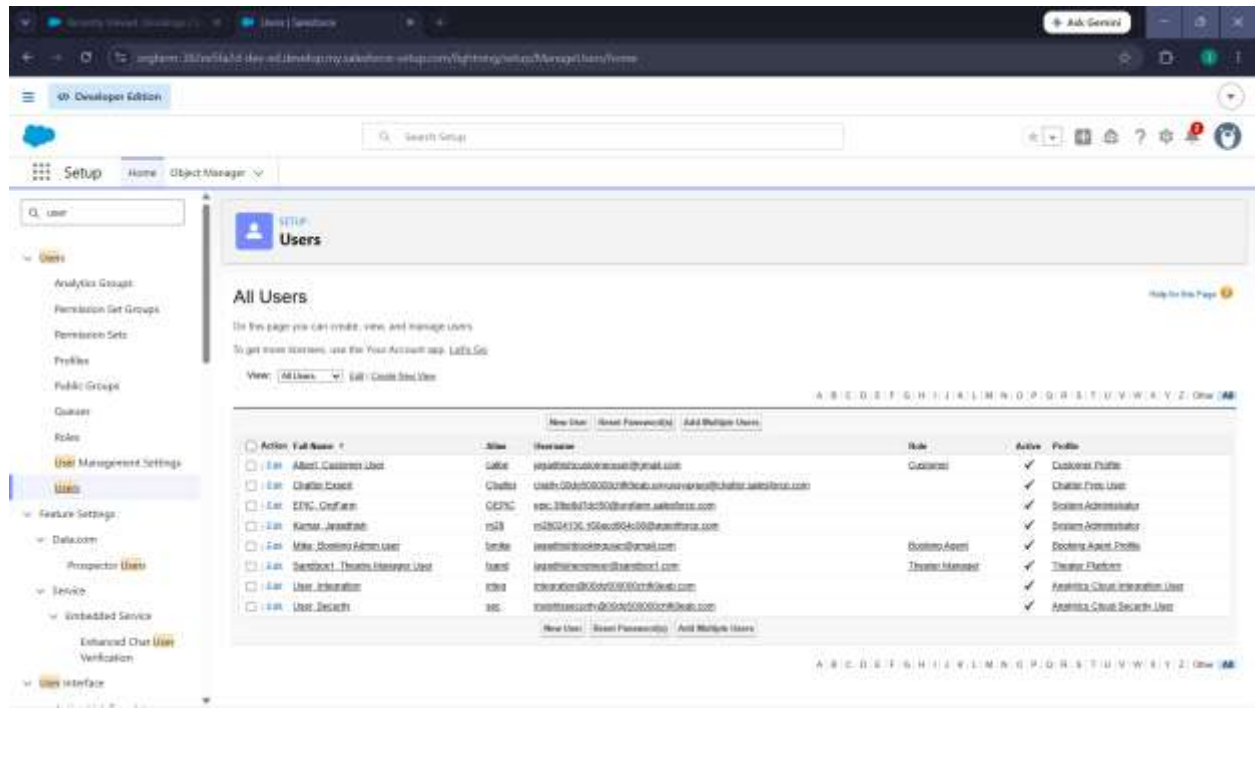
4.5 Users

Introduction

User accounts were created to test the application under different permission levels. Each user was assigned an appropriate Profile and Role.

User Management Activities

Figure 4.4 – User Management



4.6 Object-Level Security

Introduction

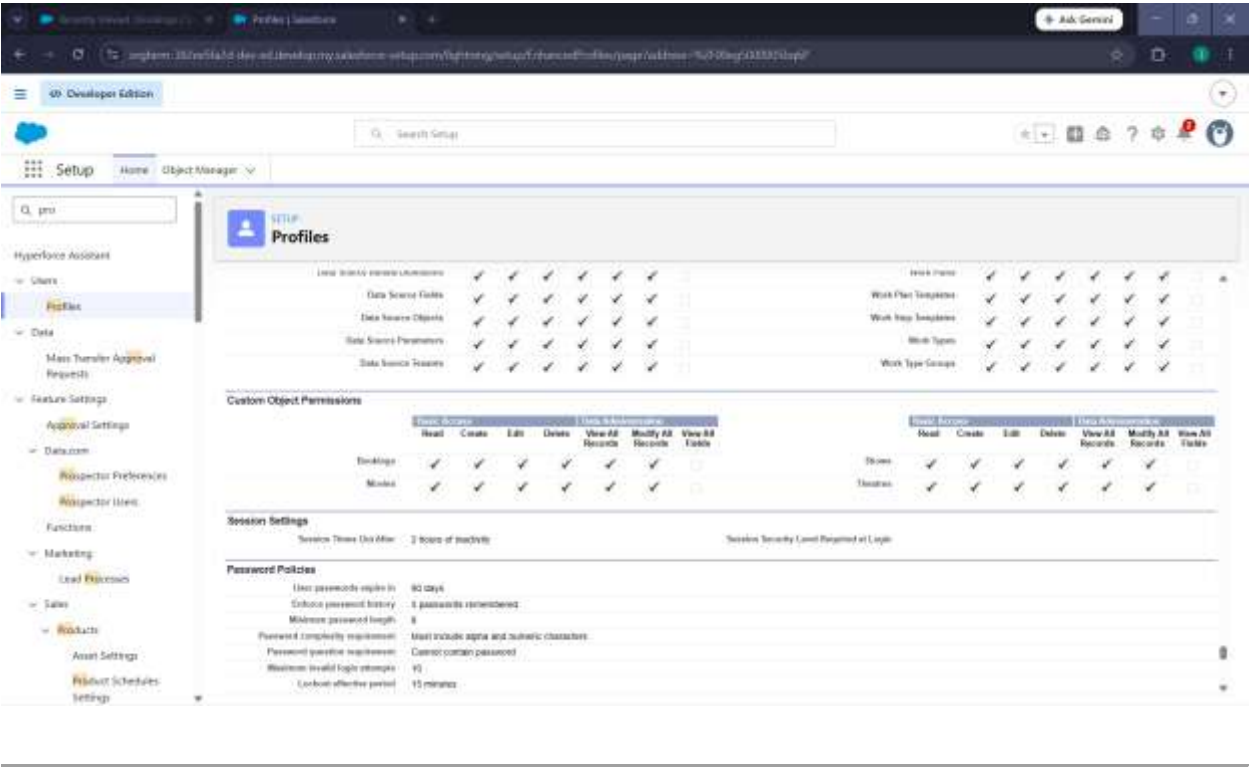
Object-Level Security controls user access to Salesforce objects through Profile permissions.

Object Permissions

Movie	Read, Create, Edit
-------	--------------------

Theatre	Read, Create, Edit
Show	Read, Create, Edit
Booking	Read, Create, Edit

Figure 4.5 – Object Permissions



4.7 Record-Level Security

Introduction

Record-Level Security controls access to individual records using Salesforce sharing mechanisms.

Sharing Mechanisms

- Organization-Wide Defaults (OWD)
- Role Hierarchy
- Sharing Rules
- Manual Sharing

Organization-Wide Defaults

Object	Default Access
Movie	Public Read Only
Theatre	Public Read Only
Show	Public Read Only
Booking	Private

[illegible]

Introduction

The application was tested to verify functionality, security, automation, Apex logic, and Agentforce integration before deployment.

Testing Objectives

- Verify business functionality
- Validate automation
- Ensure data integrity
- Confirm security
- Test Agentforce
- Verify Apex execution

The application was tested to verify functionality, security, automation, Apex logic, and Agentforce integration before deployment.

Testing Objectives

- Verify business functionality
- Validate automation
- Ensure data integrity
- Confirm security
- Test Agentforce
- Verify Apex execution

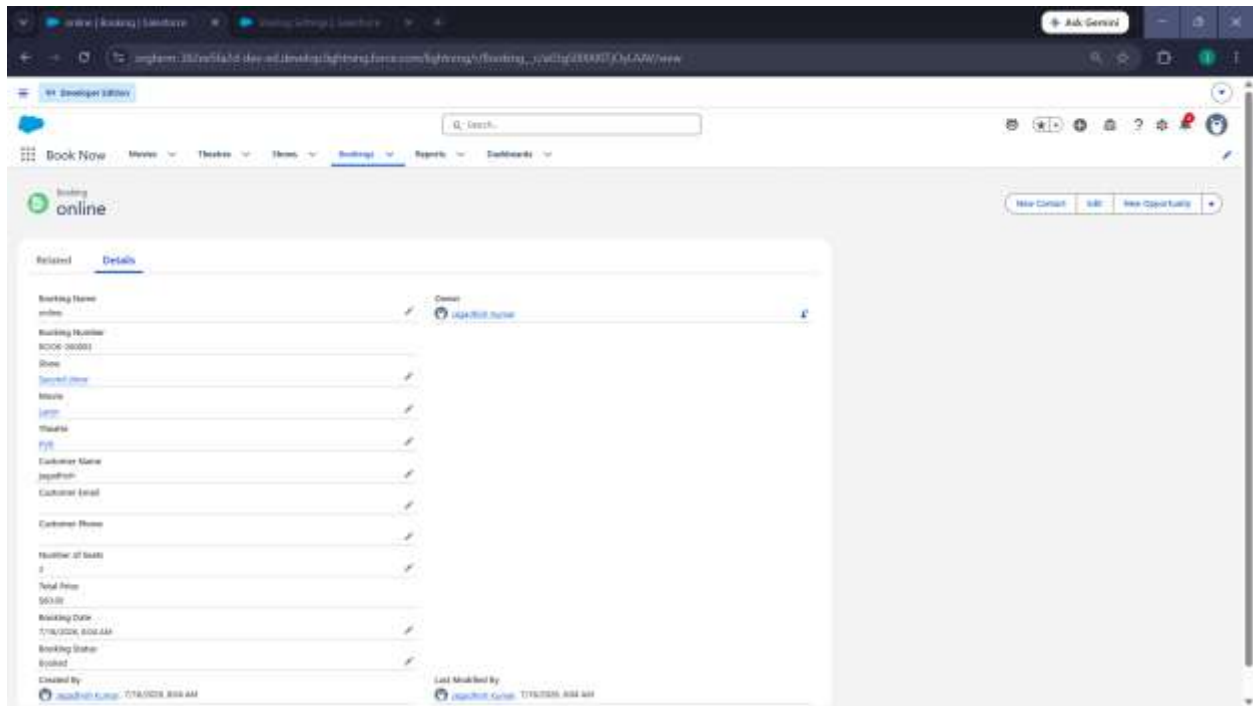
Testing Objectives

- Verify business functionality
- Validate automation
- Ensure data integrity
- Confirm security
- Test Agentforce
- Verify Apex execution

- Verify business functionality
- Validate automation
- Ensure data integrity
- Confirm security
- Test Agentforce
- Verify Apex execution

Test Scenario	Expected Result	Status
Create Movie	Record created successfully	Pass
Create Theatre	Record created successfully	Pass
Create Show	Show linked successfully	Pass
Create Booking	Booking created successfully	Pass
Update Booking	Available seats updated	Pass
Validation Rule	Invalid data rejected	Pass

Figure 4.7 – Functional Testing

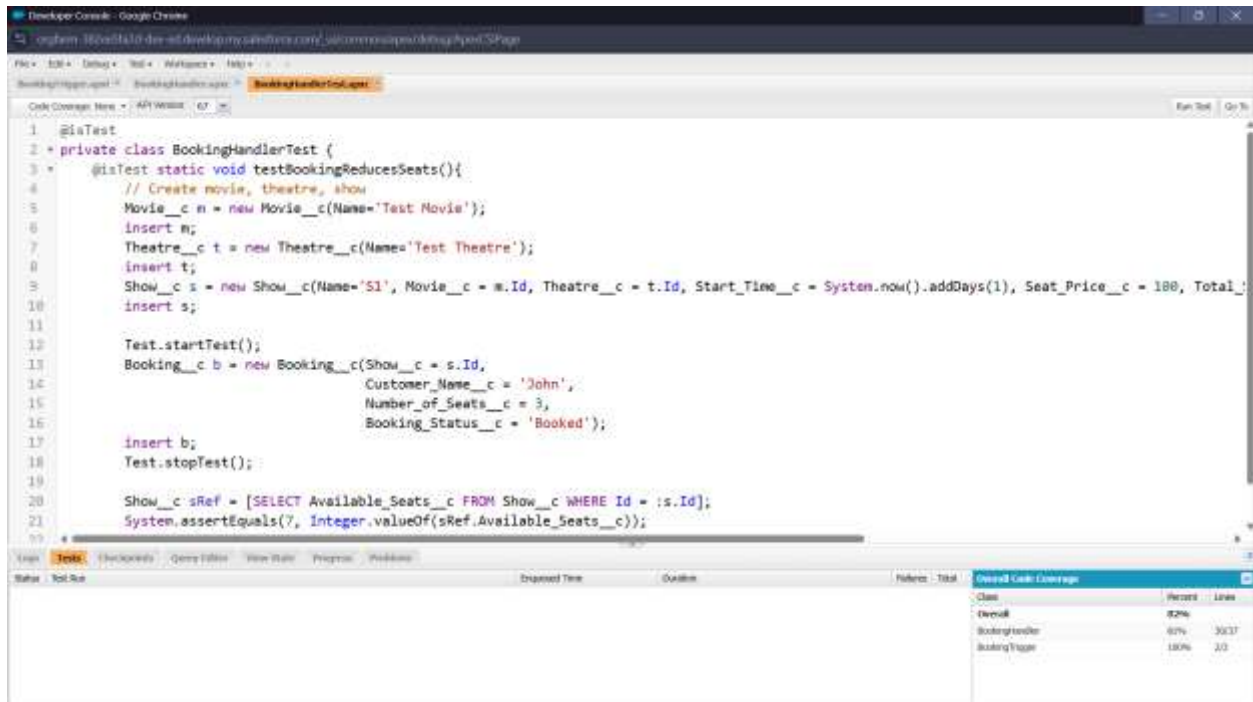


4.10 Apex Testing

The Booking Trigger and Booking Handler were tested using Apex Test Classes.

Component	Code Coverage
BookingTrigger	100%
BookingHandler	81%
Overall Project	82%

Figure 4.8 – Apex Test Results

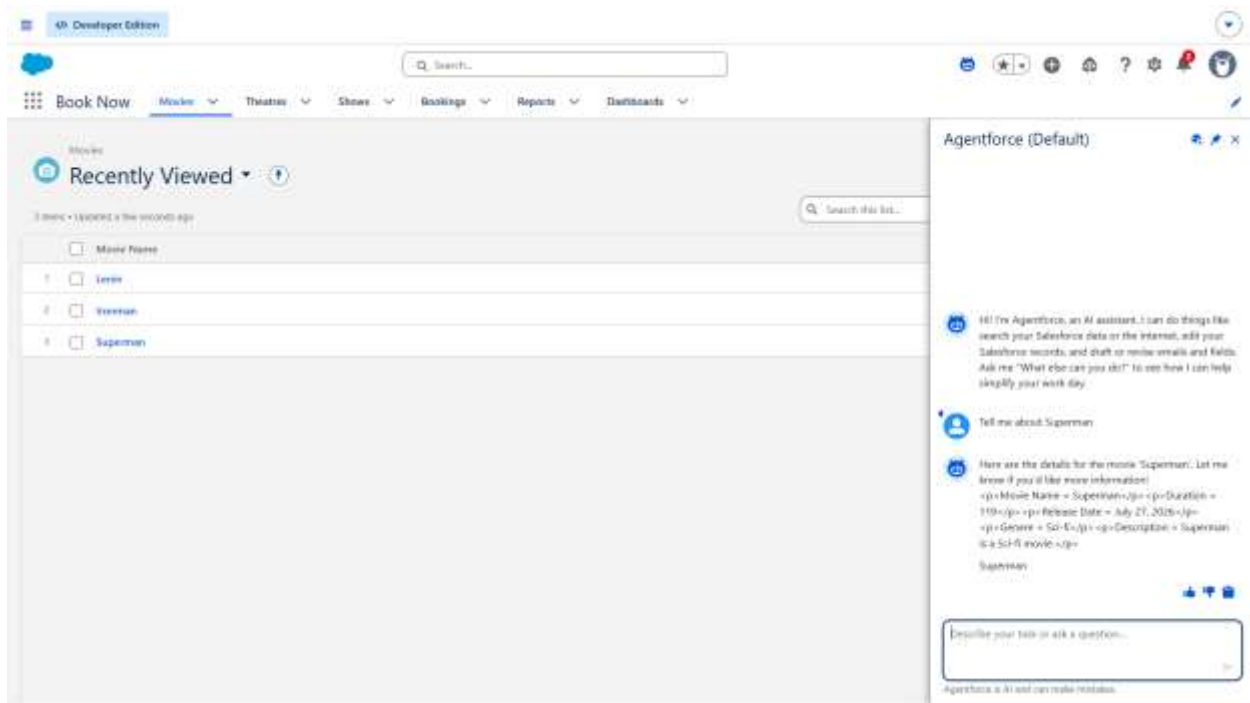


4.11 Agentforce Testing

User Prompt	Expected Result	Status
Tell me about Kalki	Movie details displayed	Pass
Movie duration	Correct duration displayed	Pass
Movie language	Correct language displayed	Pass
Release date	Correct release date displayed	Pass

Testing confirmed that Agentforce successfully retrieved movie information from Salesforce records.

Figure 4.9 – Agentforce Testing



4.12 User Acceptance Testing (UAT)

The application was evaluated from an end-user perspective.

Evaluation Results

- Navigation was intuitive.
- Booking workflow functioned correctly.
- Validation rules worked as expected.
- Reports displayed accurate data.
- Agentforce returned correct responses.

The application successfully met the defined functional requirements.

4.13 Future Enhancements

Future improvements may include:

- Online payment integration
- QR code ticket generation
- Email and SMS notifications
- Interactive seat selection
- Real-time seat locking
- Customer self-service portal
- Mobile application
- AI-based recommendations
- External movie database integration

4.14 Chapter Summary

This chapter covered Salesforce security and testing, including Profiles, Roles, Users, Object-Level Security, Record-Level Security, Functional Testing, Apex Testing, Agentforce Testing, and User Acceptance Testing. The testing results confirmed that the application functions correctly, follows Salesforce best practices, and is ready for deployment.

CHAPTER 5

Conclusion, Learning Outcomes & Future Scope

5.1 Project Conclusion

The **Book Now Movie Booking Application** was successfully developed using Salesforce declarative features and Apex programming. The application manages movies, theatres, shows, bookings, and AI-powered movie information retrieval through Agentforce. It meets the project requirements by providing a secure, scalable, and user-friendly solution with automated business processes and accurate data management.

Project Objectives Achieved

Objective	Status
Requirement Analysis	✓ Completed
Custom Objects	✓ Completed
Object Relationships	✓ Completed
Custom Fields	✓ Completed

Validation Rules	✓ Completed
Formula Fields	✓ Completed
Apex Trigger & Handler	✓ Completed
Apex Testing	✓ Completed
Lightning Application	✓ Completed
Reports & Dashboards	✓ Completed
Agentforce Integration	✓ Completed

5.2 Learning Outcomes

This project provided practical experience in:

- Salesforce Administration
- Apex Development
- Validation Rules and Formula Fields
- Salesforce Flow

- Lightning App Builder
 - Reports and Dashboards
 - Agentforce Configuration
 - Software Testing
 - Project Documentation
-

5.3 Challenges Faced

Challenge	Solution
Data Model Design	Implemented Lookup Relationships
Seat Availability	Developed Apex Trigger and Handler
Booking Validation	Applied Validation Rules
Agentforce Configuration	Configured Topics, Actions, and Flow
Application Testing	Performed Functional, Apex, and AI testing

5.4 Future Scope

Future enhancements may include:

- Online payment integration
- Interactive seat selection
- Email and SMS notifications

- Mobile application support
 - AI-based movie recommendations
 - External API integration
 - Advanced dashboards and analytics
-

5.5 Key Technologies Used

Technology	Purpose
Salesforce Lightning Platform	Application Development
Custom Objects	Data Storage
Lookup Relationships	Data Modeling
Validation Rules	Data Validation
Formula Fields	Automatic Calculations
Apex	Business Logic
Apex Trigger	Automation
Apex Test Classes	Testing

Lightning App Builder	User Interface
Reports & Dashboards	Analytics
Salesforce Flow	Automation
Agentforce	AI Integration

5.6 References

- Salesforce Documentation
 - Salesforce Trailhead
 - Salesforce Apex Developer Guide
 - Salesforce Flow Documentation
 - Salesforce Agentforce Documentation
 - Internship Project Requirement Guide
-

5.7 Acknowledgement

I sincerely thank my internship organization for providing the opportunity to develop the **Book Now Movie Booking Application**. I am grateful to my mentors and trainers for their continuous guidance and support throughout the internship. I also thank Salesforce Trailhead and the Salesforce community for providing valuable learning resources that contributed to the successful completion of this project.